

Backlog Técnico – MVP FumigApp

FumigApp es una aplicación **offline-first** dirigida a pilotos de aeroaplicación. Esto significa que todas las funciones deben operar sin conexión a Internet, almacenando datos localmente y sincronizándolos con el servidor cuando sea posible ¹ ². A continuación se detalla el backlog técnico para el MVP, organizado por épicas temáticas (Autenticación, Registro de Aplicaciones, Mapas, Sincronización, etc.), con historias de usuario orientadas al piloto. Cada historia incluye su descripción, pasos de comportamiento actual vs. esperado, criterios de aceptación paso a paso, tareas técnicas (frontend/backend) y notas de implementación con sugerencias (endpoints, hooks, componentes, almacenamiento local, etc.). Este documento está escrito en Markdown de forma clara y estructurada para facilitar su uso por un desarrollador individual.

Épica: Autenticación y Gestión de Sesión

Historia de Usuario: Inicio de Sesión del Piloto

Descripción: Como piloto aeroaplicador, quiero iniciar sesión en la aplicación con mis credenciales para acceder de forma segura a mis datos personales incluso sin conexión.

Pasos (Actual vs Esperado): 1. **Actual:** Actualmente, el piloto no cuenta con un sistema de inicio de sesión; cualquiera podría acceder a la app sin autenticación.

→ **Esperado:** Al abrir FumigApp, se debe presentar una pantalla de **login** que solicite usuario y contraseña, restringiendo el acceso solo al piloto autenticado. 2. **Actual:** Sin autenticación, los datos del piloto no están protegidos ni personalizados.

→ **Esperado:** Tras ingresar credenciales válidas y autenticarse, el piloto accede a su información y configuración personal almacenada localmente, manteniéndose disponible aun si la conexión se pierde.

Criterios de Aceptación: 1. Al abrir la aplicación por primera vez (o estando no autenticado), se muestra una pantalla de **Inicio de Sesión** con campos para usuario (o email) y contraseña, y un botón "Iniciar Sesión". 2. El piloto debe ingresar un **usuario/email** válido y una **contraseña**. Si el formato de email no es válido o faltan datos obligatorios, la interfaz mostrará un mensaje de error pidiendo corregir el formato antes de enviar. 3. Al enviar las credenciales, si estas son incorrectas (no coinciden con un usuario válido), se muestra un **mensaje de error** indicando "Usuario o contraseña incorrectos". La sesión no inicia en este caso. 4. Si las credenciales son correctas, el sistema realiza la autenticación del piloto. Esto implica: - Almacenar de forma segura un *token* de sesión o marcar al usuario como autenticado en el almacenamiento local. - Redirigir o mostrar la pantalla principal **Dashboard** una vez autenticado. 5. La pantalla de login debe ofrecer una opción "Recordarme" (checkbox). Si el piloto la activa, la aplicación conservará la sesión iniciada por un período prolongado (ej. 30 días) usando almacenamiento persistente ³ ⁴. Si no se activa, la sesión podrá almacenarse solo temporalmente (sesión actual) ⁴. 6. **Escenario offline:** Si el piloto ya inició sesión anteriormente y seleccionó "Recordarme", al abrir la app sin conexión podrá entrar directamente al Dashboard usando la sesión almacenada (si no expiró) sin requerir login de nuevo. Si la sesión expiró o no hay credenciales guardadas, y no hay conexión para validar, se debe notificar que no es posible iniciar sesión offline en ese caso. 7. Las credenciales (por seguridad) nunca se almacenan en texto plano; solo se guarda lo necesario para mantener la sesión (por ejemplo, token JWT o hash de sesión). Si la sesión expira, se forzará nuevamente la pantalla de login.

Tareas Técnicas: - Frontend:

- Desarrollar la pantalla de **Login** con formulario de usuario y contraseña, incluyendo el checkbox "Recordarme" y mensajes de error de validación.
- Implementar validaciones de formulario (formato de email válido, campos requeridos no vacíos).
- Al hacer submit, llamar a la lógica de autenticación; si es exitosa, redirigir al Dashboard. Manejar estados de carga (loading) y error en el botón de login.
- Implementar un mecanismo de **AuthGuard** en el router: las rutas de la app (Dashboard, etc.) deben estar protegidas y solo accesibles si `isAuthenticated` es true, si no, redirigir a `/login` ⁵.
- Gestionar el estado de autenticación de forma global (por ejemplo, con Context API de React para usuario y token, o mediante un hook `useAuth` personalizado).
- Si "Recordarme" está activo, almacenar el estado de sesión en `localStorage`; caso contrario en `sessionStorage`. Por ejemplo, guardar un flag `isAuthenticated=true`, datos básicos del usuario y el token en `localStorage` para sesiones persistentes ⁶.
- Asegurar que al abrir la app, si hay token/usuario en almacenamiento local y no expirado, se omita la pantalla de login y se pase directo al Dashboard.

- Backend:

- Crear un endpoint de autenticación, por ejemplo **POST** `/api/auth/login`, que reciba las credenciales y verifique usuario/contraseña. En el MVP, esto puede ser simulado localmente (ej.: credenciales demo **admin/fumigacion123** aceptadas) ⁴, ya que no hay backend real en este MVP ⁷.
- Devolver un token de sesión (p.ej. JWT) y datos del usuario (nombre, email, etc.) si la autenticación es exitosa.

- (Opcional futuro) Endpoint **POST** `/api/auth/refresh` para prolongar sesión si se usa JWT de corta duración. En MVP, dado que no hay backend real, esta lógica puede omitirse.
- Implementar en el servidor (o en la lógica simulada) validación de contraseña segura (mínimo 8 caracteres, etc.) y encriptación de la misma si correspondiese. En MVP demo, se puede simplificar.

- Notas de Implementación:

- Utilizar HTTPS/SSL (en caso de un backend real) para enviar credenciales de forma segura. En MVP local, asegurar que las credenciales se manejan solo en el cliente.
- Las credenciales de ejemplo para pruebas pueden ser **admin / fumigacion123**, tal como se usó en el prototipo ⁸, pero en producción se deberán validar contra una base de datos real.
- El almacenamiento local debe usarse con clave claras: por ejemplo, guardar `user` (datos del usuario) y `token` en `localStorage` junto con un indicador `isAuthenticated` ⁶.
- También almacenar un timestamp de último login para controlar expiración manual de 30 días (ya que en MVP no hay backend que invalide el token). La aplicación puede eliminar la sesión local automáticamente si detecta que han pasado >30 días desde el login ⁹.
- Implementar la expiración de sesión: por ejemplo, almacenar la fecha de login en `localStorage` y verificarla al iniciar; si han pasado 30 días, borrar los datos locales y exigir login de nuevo (criterio de *session cleanup* básico).
- Considerar el uso de un hook personalizado `useAuth()` para manejar login/logout, estado global y almacenamiento, o un contexto global de Auth para compartir el estado de autenticación en la app.
- Tras login exitoso, precargar en memoria los datos principales del piloto desde almacenamiento local (ej.: lista de lotes, fumigaciones) para que el Dashboard aparezca rápidamente. Si es el primer login sin datos, esas listas estarán vacías hasta que se sincronicen o ingresen manualmente.
- **Seguridad:** Aunque en el MVP no haya servidor, estructurar el código de forma que sea fácil integrar uno. Por ejemplo, aislar la lógica de autenticación (llamada a API) en un servicio o hook; así cuando haya un backend real, se reemplaza la implementación simulada por llamadas HTTP reales sin cambiar la UI.

Historia de Usuario: Cierre de Sesión (Logout)

Descripción: Como piloto, quiero poder cerrar mi sesión en la aplicación para proteger mis datos cuando termino de usarla o si otra persona va a usar mi dispositivo.

Pasos (Actual vs Esperado): 1. **Actual:** Actualmente, sin un botón de logout, la sesión del piloto permanece activa indefinidamente en el dispositivo, lo que podría permitir acceso no deseado.

→ **Esperado:** La app debe ofrecer una opción clara de "**Cerrar Sesión**" que, al usarla, elimine los datos de sesión del dispositivo y regrese a la pantalla de login, impidiendo el acceso hasta volver a autenticarse. 2. **Actual:** El piloto no tiene forma de cambiar de usuario o resetear la app sin reinstalar. → **Esperado:** Con la función de logout, el piloto (u otro usuario autorizado) puede volver a la pantalla inicial y, si es necesario, iniciar con diferentes credenciales o dejar la app bloqueada cuando no esté en uso.

Criterios de Aceptación: 1. Debe existir un elemento de UI para **Cerrar Sesión** (por ejemplo, un botón o enlace en el menú de usuario o pantalla de perfil). Es claramente identificable como opción de logout. 2. Al seleccionar "Cerrar Sesión", la aplicación solicitará confirmación (p.ej., diálogo "¿Desea cerrar su sesión?") para evitar activaciones accidentales. 3. Confirmada la acción, la app borra o invalida toda la información de sesión almacenada localmente: elimina el token de autenticación, flag de `isAuthenticated`, datos de usuario, etc., de `localStorage` / `sessionStorage`. 4. Tras limpiar la sesión, la aplicación navega inmediatamente a la pantalla de **Login**, no permitiendo acceder al contenido del piloto a menos que se vuelva a iniciar sesión. 5. (Si la app está offline en ese momento, igualmente debe permitir el logout local; la sincronización del cierre de sesión con el servidor ocurriría cuando vuelva la conexión, si aplica). 6. Cualquier intento posterior de acceder a pantallas protegidas debe redirigir nuevamente a login mientras no haya sesión iniciada.

Tareas Técnicas: - Frontend:

- Agregar un botón o menú "Cerrar Sesión" en la interfaz (por ejemplo, en el menú lateral o en la esquina superior donde aparece el nombre del piloto). Puede estar en la vista de **Perfil** o en un desplegable de usuario.

- Al hacer clic, mostrar un diálogo modal de confirmación de logout. Implementar este diálogo y su lógica de confirmación/cancelación.

- En la confirmación, ejecutar la función de logout: quitar del almacenamiento local las claves relacionadas a sesión (`user`, `token`, `isAuthenticated`, etc.) y restablecer el estado global de auth (por ejemplo, `authContext` a estado no autenticado).

- Redirigir al usuario a la ruta de login (`/login`), limpiando cualquier estado de la interfaz que pudiera permanecer (posiblemente recargando la app o usando lógica de navegación). - Asegurar que, tras logout, si el piloto intenta usar funcionalidades (p. ej., usando el historial del navegador para volver al Dashboard), el **AuthGuard** lo devuelva al login al no encontrar sesión válida. - **Backend:**

- Si existiese un backend con sesiones, implementar un endpoint **POST** `/api/auth/logout` que invalide el token o sesión del lado servidor (por ejemplo, eliminando un token de refresh o marcando un JWT como revocado en servidor). En el MVP sin backend real, esta tarea no aplica; bastará con la limpieza local.

- (Opcional futuro) Considerar invalidar todas las sesiones si se cierra sesión en un dispositivo (no aplica en MVP ya que es single-user single-device). - Asegurar que las próximas peticiones al API (si las hubiera) con ese token sean rechazadas (esto se logra guardando estado de logout en servidor, pero de nuevo, en MVP no hay llamadas reales porque es offline-first). - **Notas de Implementación:**

- **Persistencia:** Dado que es una app offline-first sin backend, el logout es principalmente local. Al diseñar, tenga en cuenta que cuando haya backend, podría mantenerse un registro de sesión en servidor; la arquitectura debería permitir enchufar una llamada a `/auth/logout` fácilmente en el flujo de logout.

- Antes de borrar datos de sesión, considerar si se deben conservar algunas preferencias del usuario (por ejemplo, el tema oscuro/claro seleccionado) – esas podrían permanecer si no están ligadas a la sesión. En este caso, solo se eliminan credenciales y datos sensibles. - Verificar que al hacer logout también se limpian/descartan los datos en memoria de la app (listas de aplicaciones, campos, etc.), o alternativamente, se puede decidir dejarlos en caché pero **ocultos** detrás del login. Para mayor seguridad, es recomendable limpiar o al menos cifrar datos locales si se quiere prevenir acceso desde fuera de la app. - Probar el flujo de logout tanto online como offline. Offline no tiene diferencias salvo

que no habrá llamada al servidor; debe funcionar igual (borrar datos y volver a login).

- UI/UX: Tras logout y retorno a pantalla de inicio de sesión, puede mostrar un pequeño mensaje tipo "Sesión finalizada" para feedback al usuario.

- Asegurar que el estado "recordado" también se borra. Si el piloto cierra sesión manualmente, aunque hubiera marcado "Recordarme", esa preferencia debe invalidarse inmediatamente (no esperar expiración). En `localStorage` esto implica borrar la entrada de sesión persistentemente guardada.

Épica: Dashboard y Reportes

Historia de Usuario: Visor de Resumen (Dashboard)

Descripción: Como piloto, quiero ver un panel de control con un resumen de mi información de fumigaciones y campos, para monitorear mi progreso y estado general de un vistazo al iniciar la aplicación.

Pasos (Actual vs Esperado): 1. **Actual:** Sin un panel de inicio, el piloto tendría que revisar manualmente listas o registros para conocer su progreso, lo que consume tiempo.

→ **Esperado:** Al iniciar sesión, la aplicación muestra un **Dashboard** con indicadores clave (número de aplicaciones realizadas, hectáreas tratadas, etc.) y accesos directos, brindando al piloto una vista general inmediata. 2. **Actual:** Navegar a distintas funciones (registrar nueva aplicación, ver mapas, etc.) requeriría varios pasos desde el menú.

→ **Esperado:** El Dashboard provee **acciones rápidas** (botones o íconos) para las tareas más comunes (ej. agregar aplicación, agregar lote, ver mapa general, consultar reportes), facilitando el flujo de trabajo del piloto.

Criterios de Aceptación: 1. Tras el login, el piloto ve la pantalla **Dashboard** principal. En esta vista se presentan al menos **4 tarjetas o estadísticas** destacadas, tales como: **Total de Aplicaciones** de fumigación realizadas, **Total de Hectáreas** tratadas, **Efectividad Promedio** de tratamientos (si se calcula) y **Aplicaciones Pendientes** (planificadas y aún no completadas) ¹⁰. Cada tarjeta muestra un número o valor claramente. 2. Los valores de estas tarjetas se obtienen de los datos locales del piloto: - *Total de aplicaciones*: cantidad de registros de fumigación marcados como completadas (o en general, número total si se cuentan todas). - *Hectáreas tratadas*: suma de las áreas de todos los lotes fumigados (o suma de áreas cubiertas en aplicaciones completadas) ¹¹ ¹². - *Efectividad*: podría representarse como un porcentaje promedio de efectividad si hay un campo para eso en los registros (si no, se puede omitir o usar otro indicador útil para el piloto). - *Pendientes*: cantidad de aplicaciones en estado Planificadas o En Proceso. (Los indicadores pueden ajustarse según los datos disponibles; lo importante es que sean significativos para el usuario). 3. El Dashboard incluye **accesos directos** (íconos o botones prominentes) para las funciones principales: por ejemplo "Nueva Aplicación", "Nuevo Lote", "Ver Mapas", "Ver Reportes", "Perfil", etc. Estos atajos permiten al piloto navegar con un solo toque a la funcionalidad deseada ¹³. Por ejemplo, un botón "+" en Aplicaciones que lleve al formulario de nueva fumigación. 4. La interfaz del panel es **responsive** y legible en exteriores: fuentes claras y suficientemente grandes, buen contraste de colores, y tarjetas dispuestas en una columna o grid adaptable según el tamaño de pantalla (pensando en tablets). 5. Los datos del Dashboard se actualizan en tiempo real dentro de la sesión: si el piloto registra una nueva aplicación o marca una como completada, las métricas correspondientes (conteos, hectáreas, pendientes) deben reflejar el cambio sin necesitar reiniciar la app. (Esto puede lograrse recalculando al volver al Dashboard o usando un estado global que se actualiza). 6. (Opcional) Cada tarjeta podría ser clicable para ver más detalle, por ejemplo: tocar la tarjeta de "Aplicaciones Pendientes" podría llevar a la lista filtrada de aplicaciones planificadas. Sin embargo, esto es un extra; no es obligatorio en MVP pero mejora la navegación.

Tareas Técnicas: - Frontend:

- Crear un componente **Dashboard** que contenga las tarjetas de métricas e íconos de acciones. Diseñar el layout para que sea claro y adaptable (por ejemplo, usando CSS Grid o flexbox para las tarjetas).
- Calcular o leer las métricas: implementar funciones que obtengan de la lista de fumigaciones local el número de completadas, pendientes, la suma de hectáreas (necesitará obtener el área de los lotes asociados o campos de hectáreas en cada registro). Puede hacerse al montar el Dashboard o mantenerse actualizado a través de un contexto o hook (e.g., un hook `useStats(fumigaciones, lotes)` que derive estas estadísticas).
- Añadir botones de **quick action**: por ejemplo, un botón con ícono de fumigador "Nueva Aplicación" que al pulsarlo lleve al formulario de nueva fumigación (navegación a esa vista), botón "Nuevo Lote" similar, ícono de "Mapa" para ir a la vista de mapa general, etc. Implementar la navegación onClick de esos botones usando el router o context adecuado. - Asegurar que los componentes de las tarjetas actualicen sus valores cuando los datos subyacentes cambien. Si se usa React, esto puede lograrse manteniendo las listas de datos en un estado global (Context o similar) y recalculando las métricas cuando ese estado cambie. - Implementar estilos teniendo en cuenta el tema claro/oscuro (por ejemplo, usar clases o ThemeContext para que las tarjetas cambien de estilo según el modo, ver **Historia de Tema** más adelante).

- Backend:

- En este MVP offline, no hay llamadas directas de backend para el dashboard; todos los datos provienen del almacenamiento local. Sin embargo, se puede planificar un endpoint agregado en el futuro, por ejemplo `GET /api/dashboard/summary`, que devuelva los totales (número de aplicaciones, etc.) para cuando exista sincronización multiusuario. Por ahora, esta lógica reside en el cliente. - Si hubiera un backend central, este endpoint podría computar estadísticas globales o del usuario, pero dado que el piloto trabaja offline en su dispositivo, las estadísticas se computan localmente para su propio registro.

- Notas de Implementación:

- **Métricas definidas:** Asegurarse de definir claramente cómo se calculan las métricas. Por ejemplo, "hectáreas tratadas" puede ser la suma de hectáreas de todos los lotes fumigados al menos una vez, o la suma total de hectáreas en todas las aplicaciones completadas (si hay repetición de campo puede contar varias veces). Decidir la interpretación que sea más útil para el piloto y documentarla. En el análisis original se mencionan estadísticas en tiempo real de *total de campos*, *total de hectáreas* y *fumigaciones activas* ¹², lo cual sugiere llevar conteos exactos. - Usar iconografía y colores representativos en cada tarjeta (por ejemplo, un ícono de avión para aplicaciones, ícono de mapa para hectáreas, etc., si ayuda visualmente). Librerías como React Icons se pueden utilizar para esto (ya se mencionó su uso en el proyecto) ¹⁴ ¹⁵. - **Quick actions:** validar que los accesos directos funcionan offline (son navegaciones locales dentro de la app, así que sí). Si alguna acción depende de internet (por ej. reportes si necesitaran datos externos), tenerlo en cuenta. En MVP, todo es local. - Revisar la persistencia: las métricas deben recalcularse tras una sincronización exitosa también, en caso de que lleguen nuevos datos del servidor (futuro). Por ahora, se recalcula tras operaciones locales. - Considerar implementar la **Pantalla de Dashboard** como un conjunto de componentes más pequeños: un componente `StatsCards` para las tarjetas de métricas, otro `QuickActions` para los botones, de modo que el código esté organizado. Esto facilita pruebas unitarias de las funciones de cálculo de stats por separado. - Enfocar la experiencia para que el piloto al abrir la app tenga inmediatamente el panorama general y los botones grandes para comenzar sus tareas del día.

Historia de Usuario: Visualizar Reportes y Gráficos Analíticos

Descripción: Como piloto, quiero ver reportes visuales (gráficos) de mis aplicaciones de fumigación para analizar la distribución de tratamientos en el tiempo y otros patrones, ayudándome a tomar decisiones y entender resultados.

Pasos (Actual vs Esperado): 1. **Actual:** Actualmente, el piloto tendría que revisar registros uno por uno y quizás anotar en papel para encontrar patrones (por ejemplo, cuántas aplicaciones hizo por mes, o

qué tipo de producto usa más). Esto es engorroso y propenso a error.

→ **Esperado:** La aplicación provee una sección de **Reportes** con gráficos automáticos que resumen la información (aplicaciones por mes, tipos de tratamientos usados, etc.), de forma comprensible y sin necesidad de cálculos manuales. 2. **Actual:** Sin reportes integrados, el piloto no dispone de estadísticas históricas visuales inmediatamente.

→ **Esperado:** Con unos pocos toques, el piloto puede visualizar gráficamente datos históricos (por ejemplo, barras por mes, porcentajes por tipo de cultivo o plaga atendida) y también filtrar esos datos según necesite ver detalles específicos.

Criterios de Aceptación: 1. La aplicación ofrece una sección o pantalla de **Reportes** (accesible desde el Dashboard mediante un botón "Reportes" o similar). Al entrar en ella, se cargan uno o varios gráficos interactivos basados en los datos de fumigación almacenados. 2. Debe haber al menos **dos tipos de gráficos** representativos: - Un gráfico de barras o de área que muestre **Fumigaciones por mes** a lo largo del año (o últimos 12 meses). Cada barra representa la cantidad de aplicaciones realizadas en ese mes ¹⁶. - Un gráfico de tipo pie (torta) o bar chart que muestre la **distribución de tipos de tratamiento o productos** utilizados ¹⁶. Por ejemplo, porcentaje de aplicaciones preventivas vs correctivas, o porcentaje de uso de herbicidas vs insecticidas, etc., según los campos *tipoTratamiento* o *producto* de los registros. 3. Los gráficos deben generarse con los datos locales del piloto. Si no hay suficientes datos (por ej., recién empieza a usar la app y tiene pocas aplicaciones registradas), el gráfico debe manejarlo graciosamente (mostrar tal vez "Insuficientes datos" o simplemente reflejar lo que haya, aunque sea 1 punto). 4. Se proveen **filtros** para refinar los reportes: - Filtro de **rango de fechas**: el piloto puede seleccionar un intervalo (ej. año actual, último trimestre, fechas personalizadas) y los gráficos se actualizarán para ese rango ¹⁷. - Filtro por **lote** específico o por **producto**: por ejemplo, ver las aplicaciones solo del "Campo Norte" o solo las que usaron "Herbicida". Al aplicar un filtro, los gráficos recalculan con esos criterios. - Filtro por **estado**: quizá ver solo completadas vs incluyendo planificadas. Esto es opcional; principalmente interesa lo realizado. 5. Los gráficos son **interactivos** en el sentido de que pueden reaccionar al hover (mostrar detalles), o permitir activar/desactivar categorías (por ejemplo, en una leyenda tocar "Fungicidas" podría ocultarlo para ver solo otros tipos, etc., si la librería lo soporta). No es obligatorio para MVP tener interactividad compleja, pero al menos debe haber leyendas claras que expliquen qué representa cada color o sección. 6. (Opcional) Un botón de **Exportar** está disponible, permitiendo obtener los datos de los reportes en CSV o PDF para compartir. *Nota:* Si bien el UI del prototipo muestra botones de exportar ¹⁸, en MVP se puede dejar estos botones deshabilitados o como placeholder si su funcionalidad no se implementa aún, pero su presencia indica al usuario que podrá exportar en el futuro. 7. La sección de reportes debe funcionar offline totalmente, dado que usa los datos ya en el dispositivo. No debe intentar cargar información de internet para mostrar gráficos (excepto quizás bibliotecas de gráficos que estén incluidas en la app). Si el piloto filtra por fechas que abarcan datos que no están en el dispositivo (no sincronizados aún), simplemente el gráfico mostrará lo disponible localmente.

Tareas Técnicas: - Frontend:

- Implementar una página o componente **ReportesAnalytics** (como se denominó en la arquitectura) para renderizar los gráficos ¹⁹. Utilizar una biblioteca de gráficos adecuada para React (en el prototipo se usó Recharts ¹⁸, se puede seguir con esa u otra equivalente).

- Preparar las **series de datos** necesarias: - Para gráfico de aplicaciones por mes: agrupar los registros de fumigación por mes/año y contar. Esto implica iterar sobre la lista local de fumigaciones, usar la fecha de cada una (campo *fechaPlanificada* o *fechaRealizada* según se considere) y sumar. Se puede hacer directamente o usando utilidades de la librería de gráficos si provee algo. - Para gráfico de distribución de tipos: agrupar por *tipoTratamiento* o por *producto* y contar porcentajes. Por ejemplo, contar cuántas fumigaciones tienen *tipoTratamiento* = "Preventiva", etc., y calcular % del total. - Desarrollar componentes de **Filtro**: - Un componente de selección de rango de fecha (puede ser dos date pickers "Desde" y "Hasta" o valores predefinidos como "Último año", etc.). Al cambiar, recomputar

datos filtrando la lista de fumigaciones por fecha dentro del rango. - Un dropdown o lista para filtrar por Lote (rellenar con los nombres de lotes disponibles) y otro para Producto o Tipo de tratamiento. Al seleccionar, aplicar filtro correspondiente (por ej., si se elige un lote "Campo Norte", filtrar registros donde lote == ese ID). - Los filtros pueden aplicarse acumulativamente (ej. filtrar por año y por lote a la vez). Implementar lógica de filtrado encadenada sobre la lista base. - Asegurar que el componente gráfico se **actualiza** cuando cambian los filtros o los datos subyacentes (usar estado interno para filtros y derivar dataset, o utilizar una librería reactiva). - Añadir botones de **Exportar CSV/PDF**: estos botones, si se presionan, deben tomar los datos filtrados actuales. Para CSV, puede simplemente construir un CSV con filas (por ejemplo: mes, cantidad) o (categoría, valor). Para PDF, podría generar una imagen del gráfico o un reporte sencillo. Dado el tiempo, quizá implementar solo **Exportar CSV** (que es más fácil: `JSON -> CSV` and download). El botón PDF se puede dejar previsto pero no funcional en MVP (o usar alguna librería si se desea). Señalar visualmente si algo no está activo ("Próximamente"). - **Backend**: - En MVP offline, no se realizan peticiones para obtener reportes; los cálculos son en el cliente. Sin embargo, plantear que en un futuro podría haber un endpoint **GET** `/api/reports?...` que devolviera estadísticas pre-calculadas, especialmente si hubieran muchos datos o para consolidación de información multiusuario. Por ahora, esta lógica es local. - Si eventualmente el backend debe soportar export, podría generarse PDF/CSV server-side para mayor fidelidad. En MVP, todo es local. - **Notas de Implementación**:

- **Biblioteca de gráficos**: Recharts (utilizada en el prototipo) es adecuada ¹⁸; permite gráficos responsivos y personalizables. Asegurarse de incluirla y probar en dispositivos de distintos tamaños. Alternativas: Chart.js, pero Recharts ya está mencionada. - **Performance**: Para las cantidades esperadas de datos en MVP (quizás decenas de registros), los cálculos son triviales. Si el historial crece a cientos, sigue siendo manejable en el cliente. Si fuera a miles en el futuro, considerar optimizaciones (como resumir datos históricos). Por ahora, enfocar en claridad. - **Efectividad promedio**: Si se quiere mostrar "efectividad", se necesita tener un dato de eficacia en cada fumigación (por ejemplo, un porcentaje de control de plaga logrado). En el MVP, no se registró explícitamente un campo así salvo quizás en *observaciones*. Se podría calcular efectividad como $(\text{completadas} / \text{planificadas}) \times 100$ si quisiéramos un indicador simple, pero esto podría confundir. Alternativamente, la tarjeta "Efectividad" del Dashboard podría omitirse o reemplazarse por otra métrica (ej. "% completadas vs planificadas"). En los reportes, no tratar "efectividad" si no hay un claro origen del dato. - **Exportar datos**: La funcionalidad de export es útil pero no vital en MVP. Si se implementa CSV, asegurarse de formatear los datos amigablemente (por ejemplo, en español usar coma decimal? probablemente no, dejar numérico simple). Para PDF, podría quedar pendiente. Documentar claramente si algún botón no está activo para que el usuario (piloto) lo sepa. - **UI de los gráficos**: usar colores distintos para cada categoría, añadir etiquetas o tooltips que muestren valores exactos cuando se toca un punto o sección. Por ejemplo, en Recharts, habilitar `<Tooltip />` y `<Legend />`. Usar términos en español en las etiquetas de ejes y leyendas (e.g., "Mes", "Cantidad de Aplicaciones"). - **Experiencia offline**: Todos los reportes funcionan sin internet; solo requieren los datos existentes. Si no hay datos en una categoría (por ejemplo ningún fungicida usado), la gráfica de productos podría omitir esa categoría o mostrar 0%. Esto está bien. - **Validar los resultados con el piloto** (si es posible) para asegurarse de que comprende los gráficos. Por ejemplo, verificar si le interesa más ver fumigaciones por mes o por campaña agrícola*, etc. En esta primera versión, mes a mes está bien.

Épica: Gestión de Lotes (Campos)

Historia de Usuario: Listar Lotes Registrados

Descripción: Como piloto, quiero ver un listado de todos mis lotes (campos) registrados en la aplicación, para poder elegir en cuál realizar una aplicación o consultar sus detalles fácilmente offline.

Pasos (Actual vs Esperado): 1. **Actual**: Sin la app, el piloto llevaría la cuenta de sus campos en papel o de memoria, pudiendo olvidar detalles o confundir nombres.

→ **Esperado:** La aplicación muestra una **lista de lotes** agrícolas gestionados, con información básica, permitiendo identificar rápidamente cada campo disponible para fumigación. 2. **Actual:** Para encontrar un campo específico, el piloto tendría que buscar en notas o mapas físicos.

→ **Esperado:** La lista de lotes permite **buscar** o filtrar campos (por nombre, por tipo de cultivo, etc.), agilizando la ubicación de un lote incluso si la lista es extensa.

Criterios de Aceptación: 1. Existe una sección "**Lotes**" en la app (accesible desde el Dashboard o menú) donde se lista cada lote registrado. Cada entrada de la lista muestra al menos: **Nombre del Lote**, tipo de cultivo, y posiblemente alguna otra info breve como área en hectáreas o localidad. Estos datos provienen de lo almacenado localmente para cada lote. 2. Los lotes en la lista aparecen ordenados de forma lógica: por defecto podría ser alfabéticamente por nombre, o quizás por fecha de creación (más reciente primero). Definir esto; alfabético suele ser intuitivo. 3. El piloto puede **buscar** un lote por nombre mediante un campo de búsqueda en esta pantalla. Mientras escribe, la lista se filtra en tiempo real mostrando solo los lotes cuyo nombre coincide con el texto ingresado (subcadena). Si ningún lote coincide, se indica "No se encontraron lotes". 4. (Opcional) Se puede también filtrar por **cultivo**: por ejemplo, un desplegable que permite seleccionar "Soja", "Maíz", etc., para ver solo lotes de ese tipo de cultivo. Esto es útil si el piloto tiene muchos campos y quiere verlos agrupados por cultivo. 5. Al seleccionar/clickear un lote de la lista, la app navega a la **vista de detalle del lote** correspondiente. 6. La lista debe cargar **offline** con los datos locales disponibles. Si el piloto aún no ha registrado ningún lote, se mostrará un mensaje del tipo "No hay lotes registrados. Use 'Agregar Lote' para crear uno.". 7. Si hay lotes precargados (por ejemplo, el MVP puede traer 10 lotes de ejemplo de Entre Ríos para demo ²⁰), estos aparecen en la lista la primera vez. 8. La UI debe ser capaz de refrescar la lista cuando se agregue, edite o elimine un lote (por ejemplo, al volver de la pantalla de "Nuevo Lote", el nuevo lote aparece en la lista sin reiniciar la app).

Tareas Técnicas: - Frontend:

- Crear el componente **LotesList** para mostrar la lista de lotes ²¹. Usar una estructura de lista con cada ítem mostrando nombre, cultivo, etc. Se puede usar una lista simple o incluso tabla si se requiere alineación (pero en móvil, lista tipo tarjeta es mejor). - Implementar el campo de **búsqueda** en la parte superior de la lista. Vincularlo a un estado `searchTerm` y filtrar la lista de lotes (por nombre principalmente). Esto puede hacerse ignorando mayúsculas/minúsculas para mayor comodidad. - (Opcional) Implementar filtro de cultivo: un dropdown lleno con todos los tipos de cultivo presentes en los lotes (por ejemplo, recopilados de la lista). Al seleccionar un cultivo, ajustar la lista mostrada. Este filtro puede combinarse con la búsqueda textual (ej: buscar "San" y filtro "Maíz" mostraría lotes de maíz cuyo nombre contiene "San"). - Asegurar que cada elemento de la lista es **clickable** (por ejemplo toda la fila o un botón "Ver") y al pulsarlo navega a la pantalla de detalle del lote (`LoteView`). - Integrar la lista con los datos locales: obtendrá la lista de lotes desde una fuente central (p. ej., un hook `useLotes()` que lee de `localStorage` o de un contexto global de datos). Si se tiene en memoria un estado global `lotes`, usarlo; si no, leer de `localStorage['lotes']` y convertir a objetos. - Implementar la capacidad de **actualización dinámica**: si un lote nuevo es añadido (en otra vista) o eliminado, el componente de lista debe actualizarse. Esto puede lograrse si la lista viene de un estado global (React state) y las operaciones de agregar/quitar lote modifican ese estado (así React re-renderiza la lista). O escuchar eventos de `localStorage` (por ejemplo, `window.onstorage`) para cambios externos ²².

²³ . - Backend:

- Preparar (para futuro) un endpoint **GET** `/api/lotes` que devuelva la lista de lotes del usuario autenticado. En MVP, como no hay backend, la app podría cargar datos iniciales de un archivo (ej.: `lotesViale.js` con datos precargados ²⁰) o simplemente empezar vacía.

- Si existiera servidor, las operaciones de CRUD de lotes (listar, agregar, editar, eliminar) tendrían endpoints correspondientes (ver historias siguientes). En esta historia, solo listar, así que GET sería el relevante. - Asegurar que el modelo de datos de lote incluye los campos necesarios (nombre, cultivo, hectáreas, etc.) para mostrarlos en la lista. (En el análisis, se definió un esquema de Lote con `id`, `nombre`,

descripción, cultivo, propietario, hectáreas, fechaCreacion, geometría, etc.)²⁴. - **Notas de Implementación:**

- **Uso offline:** La lista se poblará desde `localStorage`. Por ejemplo, la clave `'lotes'` contendrá un array JSON de objetos lote⁶. Utilizar `JSON.parse(localStorage.getItem('lotes'))` para obtenerlos inicialmente si no se tiene en memoria. Si no existe la clave, inicializarla con `[]` o con datos de ejemplo si se desea.

- Mantener la consistencia de datos: cada lote debería tener un identificador único (`id`). Si se importan lotes de ejemplo, verificar que tengan IDs únicas. Para lotes nuevos, se puede generar un UUID v4 o usar un timestamp como id. - Mostrar información relevante: por ejemplo, si el lote tiene un campo de "propietario" y el piloto maneja campos de terceros, quizás convenga mostrar "Nombre (Propietario)" en la lista. Evaluar la necesidad según usuario final. En MVP, asumir que todos son propios o que el nombre basta. - El filtro de cultivo puede implementarse sencillamente recopilando los valores únicos de `cultivo` en los lotes y haciendo un `<select>` con ellos más una opción "Todos". Si pocos lotes, podría no ser crítico, pero se incluye para evitar ambigüedad en requisitos. - **Optimización UI:** Si la lista de lotes llegara a ser muy larga (decenas), considerar hacerla scrollable eficientemente. Con 10-20 ítems no hay problema. Si hubiese 100+, quizás implementar virtualización. En MVP no se espera tal volumen. - **Integridad referencial:** Tener en cuenta que los lotes listados pueden ser usados en fumigaciones. No es función de esta historia, pero mencionar: si un lote se elimina o edita, esas operaciones deben reflejarse en los registros de fumigaciones relacionados (lo manejaremos en historias posteriores). - Probar la búsqueda con distintos casos (comienzo de nombre, medio del nombre) para asegurar que el piloto encuentra "Estancia San Jorge" tanto escribiendo "San" como "Jor". - UI/UX: Si la pantalla es pequeña, quizás en vez de tabla, mostrar cada lote como una "tarjeta" con líneas: Nombre, Cultivo, etc., para mayor claridad táctil. En tablet es manejable una lista tipo tabla también. Decidir según se prefiera consistencia con otros diseños.

Historia de Usuario: Ver Detalle de un Lote

Descripción: Como piloto, quiero ver la información detallada de un lote específico, incluyendo su ubicación en el mapa y sus características, para consultar sus datos y verificar el terreno antes de realizar una aplicación.

Pasos (Actual vs Esperado): 1. **Actual:** Sin esta función, el piloto tendría que recordar o buscar en archivos físicos los detalles (tamaño, cultivo, etc.) de un campo, y usar mapas en papel para visualizar su extensión.

→ **Esperado:** Al seleccionar un lote en la app, se muestra una **pantalla de detalle** con todos los datos almacenados de ese lote y un mapa que delinea el campo, permitiendo al piloto tener toda la información en un solo lugar. 2. **Actual:** Para estimar áreas o comunicar detalles de un lote, el piloto posiblemente hace cálculos manuales de hectáreas o dibujos.

→ **Esperado:** La vista de detalle del lote muestra el área ya calculada en hectáreas y la forma exacta del lote en un mapa digital, eliminando conjeturas y potenciales errores.

Criterios de Aceptación: 1. Al elegir un lote de la lista de lotes, la app navega a la **Vista de Detalle** de ese lote. En esa vista se muestran: - **Datos textuales:** Nombre del lote, tipo de cultivo, propietario (si aplica), superficie (hectáreas) del lote, fecha de creación/registro del lote, y cualquier otra propiedad relevante (p.ej., descripción o notas del lote si existieran). - **Mapa del lote:** un recuadro o sección con un mapa que muestra la ubicación y contorno (*polígono*) del lote. Debe estar centrado en el lote y mostrar sus límites claramente delineados. 2. El **área en hectáreas** del lote se presenta junto a los datos. Este valor puede provenir del cálculo realizado cuando se definió el polígono. Debe ser consistente con la geometría dibujada (si el polígono se actualiza, recalcular el área). Por ejemplo: "Superficie: 25.4 ha". 3. En el mapa, el lote se muestra como un polígono resaltado (por ejemplo, borde verde semitransparente rellenando el área). Puede incluir un marcador o etiqueta con el nombre del lote sobre el polígono para

identificarlo visualmente. 4. El mapa debe funcionar offline: los **tiles** (imágenes de mapa base) deberían ser de OpenStreetMap u otra fuente abierta ²⁵. Si el dispositivo no tiene conexión, podría suceder que el mapa base no cargue completamente; aun así, el polígono del lote *debe* dibujarse sobre el fondo (incluso si está gris). En situaciones ideales, el piloto habrá abierto ese mapa online alguna vez y quedó en caché, pero no se depende de eso para la funcionalidad principal (el contorno del lote). 5. La vista de detalle ofrece opciones de acción: - Un botón para **Editar** el lote (ver siguiente historia) y posiblemente un botón para **Eliminar** el lote (ver historia correspondiente). Estos deben estar accesibles, por ejemplo, en la esquina superior (ícono de lápiz para editar, ícono de papelera para eliminar). - (Opcional) Un botón "Ver en mapa general" que abre la vista de mapa con este lote resaltado, si el piloto quiere verlo en contexto con otros campos. 6. Toda la información se presenta de manera **solo lectura** (hasta que se presione editar). El piloto puede confiar en que estos datos son exactamente los que se registraron. 7. **Integración con aplicaciones:** Si es útil, podría listar en esta pantalla las últimas aplicaciones de fumigación realizadas en este lote (por ejemplo, "Última fumigación: 10/10/2025 - Herbicida"). Esto es adicional y no obligatorio en MVP, pero aportaría valor. Si se incluye, se puede mostrar una pequeña sección "Historial en este lote" con, digamos, las 1-3 fumigaciones más recientes (fecha y tipo). Sino, se omite por simplicidad.

Tareas Técnicas: - Frontend:

- Construir componente **LoteView** para la pantalla de detalle del lote ²¹. - Obtener la información del lote seleccionado: esto vendrá probablemente vía router (p.ej., `<Route path="/lotes/:id">` pasando el `id`), o mediante un estado global seleccionado. Usar ese ID para buscar el objeto lote en la lista de lotes (en localStorage o contexto) y así obtener sus datos. - Renderizar los campos de información textual en un diseño claro (tal vez una lista vertical: Nombre, Cultivo, Hectáreas, etc.). Incluir unidades en los valores (ha, si muestra costo o otro valor, etc.). - Implementar el **mapa embebido**: utilizar **Leaflet** (a través de React-Leaflet) para mostrar un mapa centrado. Configurar el componente `MapView` (como sugiere la arquitectura) pasándole la geometría GeoJSON del lote para que dibuje el polígono ²⁶. - Agregar una capa de mosaico (tile layer) de OpenStreetMap para el mapa base (URL estándar). - Dibujar el polígono: se puede usar `L.geoJSON(geometria)` para añadirlo, o en React-Leaflet usar `<GeoJSON data={geometria} />` con estilo personalizado (bordes verdes, relleno semitransparente). - Calcular el bounds del polígono para hacer `map.fitBounds(bounds)` y asegurar que el lote se ve completo en pantalla. - Asegurar que Leaflet no intente acceder a internet si no hay conexión más que para los tiles (los tiles pueden faltar offline, pero el polygon se verá). Opcional: podrían utilizar un tile de cache o un estilo offline (por ej., un fondo liso con coordenadas). - Añadir los botones **Editar** y **Eliminar**: - El botón Editar navegará a la pantalla de edición de lote (reutilizar el formulario de creación en modo edición). - El botón Eliminar abrirá un diálogo de confirmación y, si se confirma, ejecutará la lógica de borrado (ver historia Eliminar Lote). - (Opcional) Si se decide mostrar historial de fumigaciones en este lote, implementar una pequeña sección que filtre las fumigaciones locales por `loteId == actual`. Mostrar quizás en texto: "Aplicaciones registradas: 5 (Última el 10/10/2025)". Esto sería una sub-tarea extra. - **Backend:** - Preparar un endpoint **GET** `/api/lotes/{id}` que devuelva los detalles completos de un lote, incluyendo su geometría. En MVP no se usará realmente (datos ya locales), pero definirlo para futuro. - Si el mapa base offline es un tema, se podría pensar en un servicio de mapas offline (no trivial). En entorno con backend, podría haber un proxy de tiles o incluso mapas vectoriales precargados. En MVP, simplemente se utiliza OSM en línea con la esperanza de cache. - **Notas de Implementación:** - **Cálculo de área:** Asegurarse de que el área mostrada (hectáreas) corresponde al polígono. Si la geometría fue dibujada en la app, idealmente se calculó al guardar. Si no, se puede calcular on-the-fly en esta pantalla utilizando Turf.js (`turf.area(geoJSON)` para m² y convertir a ha) ²⁷. Para evitar diferencias, es mejor almacenar el valor en el objeto `lote` cuando se crea y solo recalcularlo si la geometría cambió. - **Map appearance:** Aplicar estilos al polígono para que sea visible sobre cualquier fondo (e.g., borde de 2px en color verde oscuro y relleno verde claro con opacidad ~0.4). Estos colores pueden integrarse al tema (ej., usar un color agronómico como #4a7c59 de la paleta mencionada ²⁸). -

Offline maps: Considerar la posibilidad de usar un proveedor de mapas offline. Por ejemplo, Mapbox puede predescargar, pero requiere configuración. En MVP quizás no se haga nada especial, solo advertir que sin conexión el fondo puede no aparecer. En las notas al usuario, se podría recomendar abrir el mapa una vez con internet para cache si va a campo sin señal. - **Editar/Eliminar permisos:** Dado que es un sistema single-user en MVP, se asume el piloto puede editar o borrar sus lotes libremente. No obstante, en futuras multiusuario con roles podría haber restricciones (por ejemplo, operarios sin permiso de borrar). No aplicar tales restricciones ahora. - **Integración con fumigaciones:** Si se llegara a mostrar resumen de actividades en el lote, hay que cuidar actualizarlo si se agregan nuevas fumigaciones en ese lote. Como es opcional, documentar que no es crítico de implementarlo ahora. - Probar la vista con distintos tamaños de polígono (lotes muy grandes vs pequeños) para ver cómo se comporta el ajuste de zoom del mapa. - UI/UX: Permitir zoom manual en el mapa y desplazamiento, para que el piloto pueda inspeccionar más de cerca si quiere. Agregar un control de zoom +/- en la esquina (Leaflet tiene por defecto). - Confirmar que los datos textuales se actualizan si el lote es editado (ej., nombre cambiado). Esto dependerá de volver a esta vista tras edición; se puede refrescar directamente leyendo desde la fuente actualizada.

Historia de Usuario: Registrar Nuevo Lote

Descripción: Como piloto, quiero **agregar un nuevo lote** (campo) a la aplicación, ingresando sus datos y dibujando su límite en el mapa, para que quede registrado y disponible offline al planificar o documentar mis aplicaciones.

Pasos (Actual vs Esperado): 1. **Actual:** Si el piloto adquiere o identifica un nuevo campo para fumigar, actualmente tendría que anotarlo en papel o recordarlo, sin tenerlo en la app. Esto puede llevar a que no tenga los datos a mano al trabajar.

→ **Esperado:** El piloto puede crear inmediatamente un registro del nuevo lote en la app: asignarle un nombre, especificar el cultivo y otros datos, y delimitar el área geográfica en un mapa dentro de la aplicación. Así el lote queda guardado en su dispositivo para futuras consultas y aplicaciones, incluso sin señal. 2. **Actual:** Calcular el área de un lote nuevo requeriría herramientas externas o cálculos manuales.

→ **Esperado:** Al dibujar el polígono del lote en el mapa durante el registro, la app calcula automáticamente la superficie en hectáreas y la almacena junto con el lote, dando al piloto esa información instantáneamente.

Criterios de Aceptación: 1. En la sección Lotes, el piloto tiene un botón "**Agregar Lote**" o similar. Al pulsarlo, se abre la pantalla o formulario de **Nuevo Lote**. 2. El formulario de creación de lote solicita los siguientes datos: - **Nombre del lote:** (Texto, obligatorio) Nombre identificador del campo. Debe ser único al menos para el usuario (no dos lotes con el mismo nombre idealmente, aunque no es estrictamente técnico, pero se puede advertir si repite). - **Cultivo:** (Selección, obligatorio) Tipo de cultivo principal del lote. Ej.: Soja, Maíz, Trigo, Sorgo, etc. Presentar una lista desplegable con las opciones comunes. - **Propietario:** (Texto, opcional) Nombre del propietario o estancia, en caso de que el piloto trabaje para terceros o quiera esa referencia. - **Descripción/Notas:** (Texto largo, opcional) Cualquier detalle adicional del campo (ej. "Lote propenso a encharcamiento en el sur", etc.). - (El campo de hectáreas no se pide manualmente, será calculado tras dibujar la geometría). - **Delinear en Mapa:** Una sección donde el piloto define la ubicación y forma del lote: - Muestra un mapa (centrado quizá en una ubicación por defecto o última conocida) con herramientas de dibujo de polígono. Instruye al piloto: "Dibuje el contorno del lote en el mapa". - El piloto puede agregar puntos en el mapa formando el polígono que representa el lote. Debe poder cerrar el polígono (la herramienta debería ayudar a ello, normalmente con leaflet-draw se hace automáticamente). - Si comete un error, debe haber opción de *deshacer* el último punto o *borrar* y volver a dibujar. 3. La aplicación **calcula automáticamente** el área del polígono dibujado en el mapa. Una vez cerrado el polígono, se muestra en el formulario el valor

calculado: "Área: X hectáreas". Este valor se guardará junto con el lote ²⁷. Debe tener al menos dos decimales de precisión o un redondeo razonable. 4. **Validaciones:** El formulario no permite guardar si: - No se proporcionó nombre (obligatorio). - No se seleccionó cultivo (obligatorio). - No se ha dibujado el polígono en el mapa (obligatorio, porque sin ubicación no tendría sentido). Si falta alguno de estos, al intentar guardar, se muestra un mensaje apropiado junto al campo faltante ("Por favor ingrese un nombre", "Seleccione un cultivo", "Debe delinear el lote en el mapa"). 5. Al pulsar **Guardar/Crear**, si todo es válido: - Se crea un nuevo objeto Lote en la base de datos local: se le asigna un `id` único (p.ej., un UUID o concatenación de nombre+timestamp). - Sus datos (nombre, cultivo, propietario, descripción, hectáreas, geometría en formato GeoJSON, fechaCreacion) se almacenan en el array de lotes en `localStorage` y/o en el estado global de la app. - La app navega de vuelta a la lista de lotes o bien muestra directamente la vista de detalle del lote recién creado. En cualquier caso, el usuario recibe confirmación de que el lote fue guardado (puede ser un pequeño mensaje "Lote guardado exitosamente"). 6. El nuevo lote aparece ahora en la **lista de lotes** (y también estará disponible en el selector de lotes al registrar fumigaciones). 7. **Offline:** Todo este proceso ocurre sin internet. Si más adelante se sincroniza con un servidor, el lote se deberá enviar, pero eso se maneja en la épica de Sincronización. En el contexto inmediato, el lote está solo en el dispositivo pero plenamente utilizable.

Tareas Técnicas: - Frontend:

- Crear el componente **LoteForm** para el formulario de creación/edición de lote ²¹. Este componente se reutilizará para editar, por lo que conviene diseñarlo genérico. Inicialmente, en modo "crear", los campos están vacíos. - Implementar los campos de entrada: usar controles adecuados (input text para nombre, select `<select>` para cultivo con opciones estáticas de cultivos comunes, textarea para descripción, etc.). Aplicar atributos de required donde corresponda e indicadores visuales para obligatorio. - Integrar un componente de **mapa con dibujo**: se puede utilizar la librería **react-leaflet** junto con el plugin **Leaflet.Draw** para habilitar dibujo de polígonos ²⁹ ³⁰. Esto requerirá incluir los scripts y estilos de leaflet-draw. - Al montar el mapa en este formulario, instanciar la herramienta de dibujo: por ejemplo, un botón "Dibujar polígono". Al activarlo, el piloto puede marcar puntos en el mapa. - Escuchar el evento de finalizado de dibujo (Leaflet Draw lanza un evento `draw:created` con la capa dibujada). En ese evento: * Obtener la geometría del polígono creado (convertir a GeoJSON). * Calcular el área: usando Turf.js `area()` con esa GeoJSON ³¹, convertir a hectáreas (dividir por 10000). * Actualizar el estado del formulario: guardar la geometría y el área calculada. Mostrar el área en un campo de solo lectura o en un texto informativo en el formulario. * Remover la capacidad de dibujar otro polígono o deshabilitar el botón de dibujar una vez uno está presente, para evitar múltiples en un mismo lote. Quizás permitir editar el polígono dibujado (Leaflet.Draw tiene modo de edición) si el usuario quiere ajustar vértices. - Si el usuario quiere rehacer, se puede ofrecer un botón "Borrar polígono" que elimine la capa actual y permita dibujar de nuevo. - Validar los campos en la función de submit del formulario: si faltan datos, evitar la creación y mostrar errores. También validar que la geometría existe (se puede requerir guardarla en estado `geom` y chequear que no esté null). - Al confirmar guardado: * Preparar un objeto `lote` con la estructura definida (generar id, tomar campos del formulario, incluir `geom` GeoJSON y área). * Llamar a la lógica de almacenamiento: p.ej., usar el hook/useLotes context para añadir el lote a la lista y actualizar `localStorage`. Si no hay tal contexto, directamente obtener la lista actual de `localStorage`, hacer push, y guardar de nuevo. * Manejar errores poco probables (por ejemplo, falta de espacio en localStorage, o si el JSON es malformado). * Finalmente, redirigir: probablemente volver automáticamente a la pantalla de lista de lotes, donde debería aparecer el nuevo. O mostrar la vista detalle de ese lote. La UX puede ser cualquiera de las dos; elegir una (mostrar detalle puede dar feedback inmediato con el mapa final, mientras volver a lista permite seguir agregando otros). - UI: añadir un botón "Cancelar" en el formulario por si el piloto decide no crear el lote. Este debe descartar lo hecho (si dibujó algo no guardado, simplemente no persistirá) y volver a la lista sin cambios. - **Backend:**

- Implementar un endpoint **POST** `/api/lotes` para recibir nuevos lotes. En un escenario con servidor, este endpoint tomaría los datos (incluyendo geometría) y asignaría un id (que podría devolver en la

respuesta). - En offline MVP, esta llamada no se realiza inmediatamente. En su lugar, el lote se marca para **sincronización pendiente**. Quizá agregar una bandera `syncPending: true` en el objeto o mantener una lista local de acciones pendientes (ver Sincronización). Así, cuando haya conexión, se enviará. - No obstante, si asumimos single-user offline MVP con posterior sync manual, se puede simplemente guardar local y dejar la sync para después. - Si existiera backend, considerar formato de enviar la geometría (GeoJSON es un buen formato, enviar como JSON). - **Notas de Implementación:**

- **Dibujo en mapa:** Este es el punto crítico de esta historia. Leaflet Draw provee la UI estandarizada (botón de polígono) que los usuarios pueden entender con un pequeño entrenamiento. Incluir instrucciones al usuario como parte de la UI: por ejemplo, un texto "Toque el ícono de polígono y marque los puntos en el mapa para delinear el lote. Al finalizar, cierre la forma y presione guardar." -
- **Geometría y Proyección:** Asegurarse de usar las mismas coordenadas para área. Leaflet entrega lat/lon en WGS84; el cálculo de área con Turf en esas coords está bien para aproximación, pero para mayor exactitud se podría proyectar a UTM. Sin embargo, dado que la extensión no es enorme (lotes ~20-50 ha) el error es mínimo. Turf area con GeoJSON lon/lat estará en m² de superficie esférica. Es aceptable en MVP (los errores serán menores al 1-2%). Documentar que se asume esa precisión. -
- **Capacidad de Edición del polígono:** Leaflet Draw también permite reabrir un polígono y mover vértices. Se podría habilitar en el mismo formulario antes de guardar, por si el piloto necesita ajustar. Si es complejo, al menos permitir borrar y volver a dibujar es suficiente. -
- **Experiencia de dibujo:** Si el piloto está en el campo sin conexión, el mapa de fondo puede no cargar. Debe poder dibujar sobre un fondo en blanco. Quizá se podrían mostrar solo las coordenadas de lat/lon mientras dibuja de referencia. Esto es un caso extremo; quizás el piloto registrará nuevos lotes cuando tenga conexión o cargó previamente la zona en caché. -
- Prever que algunos pilotos podrían querer ingresar coordenadas exactas en lugar de dibujar (por ahora eso queda fuera de alcance; el dibujo es más visual). -
- **Identificador único:** Para id del lote, se puede usar una librería de UUID. O una combinación de nombre normalizada + timestamp. Lo importante es que no choque con otros. Dado que local no hay riesgo de colisión a menos que se sincronicen con servidor (en cuyo caso servidor podría reasignar). Para MVP, un UUID v4 string es seguro. -
- Tras guardar, la **lista de lotes** deberá reflejar el nuevo. Si usamos un estado global, quizás el `addLote` ya cause re-render. Si no, tras `localStorage.setItem` se puede forzar re-fetch de la lista. -
- Verificar que la suma de hectáreas en Dashboard (si se muestra) ahora incluye este nuevo lote en el total de campos/área. -
- **Uso futuro de geometría:** Al guardar la geometría, asegurarse de almacenarla en un formato consistente (GeoJSON Feature). Así se reutiliza fácilmente en mapas (como en detalle o mapa general). Por ejemplo, almacenar `{ id, nombre, cultivo, ..., geometria: { type: 'Feature', properties:{}, geometry:{ type:'Polygon', coordinates:[[...]] } } }`. -
- Probar la creación con distintos polígonos: concavo, convexo, muchos puntos. Turf calcula bien cualquiera. Tener cuidado con polígonos auto-intersectantes: idealmente, el dibujo debería prevenir eso (Leaflet Draw usualmente no permite intersección fácil, pero no garantiza). Podríamos utilizar `turf.booleanValid()` si existiera para validar, pero en MVP tal vez no profundizar. Solo nota: lotes típicos son convexos. -
- Si hay límite en localStorage (~5MB), guardar geometrías muy complejas (muchos vértices) podría consumir espacio. Un lote poligonal normal con digamos 20 puntos es ligero. No problem, pero un campo irregular con 100+ puntos podría pesar. No es común, pero a mencionar que el almacenamiento es finito ³². En caso extremo, simplificar la geometría (Leaflet Draw no lo hace automáticamente, pero se podría aplicar una simplificación con Turf).

Historia de Usuario: Editar Lote Existente

Descripción: Como piloto, quiero poder **editar la información de un lote** que ya tengo registrado (por ejemplo cambiar su nombre, actualizar el cultivo o redefinir el área) para corregir errores o reflejar cambios (rotación de cultivo, ampliación del campo, etc.).

Pasos (Actual vs Esperado): 1. **Actual:** Si algún dato de un lote cambia (nuevo cultivo en la temporada, o se detectó que el área dibujada estaba mal), actualmente el piloto no tiene manera de actualizar la

info en la app, causando que los datos queden desactualizados.

→ **Esperado:** La aplicación permite entrar en modo edición de un lote donde el piloto puede modificar campos textuales (nombre, cultivo, etc.) y también redibujar el polígono si es necesario, guardando los cambios para mantener los registros al día. 2. **Actual:** Podría haber inconsistencias sin edición (ej., el nombre de lote cambió en la realidad pero en la app sigue el viejo).

→ **Esperado:** Con la edición, los datos de la app reflejan la realidad actual: los nombres y atributos de los lotes son correctos, lo que evita confusiones al registrar fumigaciones.

Criterios de Aceptación: 1. En la vista de detalle de un lote, al pulsar el botón **Editar**, se abre el mismo formulario de Lote (vista de edición) pero pre-llenado con los datos existentes del lote seleccionado. 2. El piloto puede cambiar cualquier campo editable: - Nombre, cultivo, propietario, descripción: pueden ser editados simplemente cambiando el texto o selección. - **Geometría:** se debe permitir al piloto actualizar el polígono del lote si, por ejemplo, hubo un cambio de alambrado o si corrigió la delimitación. Esto podría hacerse de dos formas: - Permitir que en el modo edición, el polígono dibujado actualmente aparezca en el mapa y sea *editable* (arrastrando vértices) usando la herramienta de edición de Leaflet Draw. - O, más simple pero menos sofisticado, permitir borrar el polígono actual y dibujar uno nuevo desde cero (como si re-definiera el lote). - En todo caso, el formulario debe mostrar el polígono actual inicialmente para que el piloto decida si retocarlo o dejarlo. 3. Validaciones similares a creación: no permitir dejar nombre vacío, etc. Además, si cambia la geometría, recalcular el área y actualizar el campo de hectáreas mostrado. 4. Al guardar los cambios: - La información del lote en localStorage/estado se actualiza con los nuevos valores. Si se modificó el nombre, ese nombre se reflejará en la lista de lotes, en cualquier fumigación asociada (posiblemente en su referencia se guarda id, así que realmente no cambia en fumigación salvo que se mostrara el nombre). - Si se modificó el polígono, la nueva geometría y hectáreas reemplazan a la anterior en el almacenamiento. - La fecha de actualización podría registrarse (por ejemplo, `updatedAt` del lote) si se quiere llevar control; no obligatorio pero útil para sync o logs. - La app vuelve a la vista de detalle actualizada (o a la lista de lotes con el elemento actualizado). Lo importante es que el piloto vea confirmados los cambios (p.ej., en la vista de detalle ya con datos nuevos). 5. Si el piloto cancela la edición (hay un botón "Cancelar" en edición), se descartan los cambios y se regresa a la vista detalle sin alteraciones. 6. Consideraciones offline: la edición se hace localmente; si hay sync luego, se debe marcar este lote como modificado para enviar al server. Pero en la usabilidad local inmediata, los cambios son efectivos enseguida.

Tareas Técnicas: - Frontend:

- Reutilizar el componente **LoteForm** para modo edición. Al entrar en modo edición, cargar en el estado inicial del formulario los valores del lote actual (props o context pueden proveer el objeto lote a editar). - Para la geometría, inicializar el mapa con el polígono existente ya dibujado. Usar Leaflet Draw en modo edición: * Agregar la capa existente. * Activar `editMode` en Leaflet Draw, que permitirá mover vértices y agregar/eliminar vértices. * Alternativamente, si editar vértices es complicado de implementar, se puede dar opción "Redibujar Lote": al pulsar, borra el polígono actual y activa la herramienta de dibujar uno nuevo. Esto reinicia la geometría. (Menos preciso pero más fácil de implementar sin profundizar en Leaflet Draw's edit). * Mejor experiencia es usar el modo edición de la propia librería: el piloto arrastra puntos para ajustar. Implementar escuchando `draw:edited` evento para cuando termine de editar, recalcular área. - Propagar los cambios: cuando el piloto modifica texto o selección, simplemente se cambian en el estado; la lógica de validación y guardado es la misma que en crear (chechar requeridos, etc.). - Al hacer submit en edición: * Encontrar el lote en la lista global (por id) y actualizar sus campos. Si se usa context/hook, crear un método `updateLote(id, newData)`. * Actualizar `localStorage['lotes']` con el array modificado. * Navegar de vuelta a la vista detalle (o lista). * Mostrar algún indicador de éxito ("Lote actualizado"). - Asegurar que la lista de lotes reactive el cambio (por ejemplo, si nombre cambió, la lista debe mostrar el nuevo nombre). Con estado global, esto sucede automáticamente. Sin estado global, podría requerir recargar la lista; quizás no sea un problema si al volver de edición se re-lee de storage. - **Backend:**

- Prever un endpoint **PUT** o **PATCH** `/api/lotes/{id}` para enviar los campos cambiados de un lote. En offline MVP, este no se invoca aún, pero se debe marcar el lote local como "pendiente de actualización" para la sincronización (por ejemplo, mantener un campo `updatedAt` y comparar con server, o un flag `needsSync`). - Decidir en la sincronización cómo manejar ediciones concurrentes (no aplica en single-user, pero multi-user en el futuro podría requerir merge o última escritura gana). -

Notas de Implementación:

- **Integridad con fumigaciones:** Si se cambia el nombre del lote, y las fumigaciones almacenan referencia por ID (lo cual es lo esperable), no hay que modificar nada en ellas ya que siguen apuntando por id. Sin embargo, si en la UI de fumigaciones se muestra el nombre del lote, conviene que al actualizar el nombre de un lote, cualquier lista de fumigaciones o detalle de fumigación que muestre el nombre refleje el cambio. Esto se logra fácilmente si se hace búsqueda por id en render; si se almacena una copia del nombre dentro de cada fumigación, entonces habría inconsistencia. Mejor no duplicar nombre en fumigación, usar id para referencia. - **Rotación de cultivo:** Si el cultivo de un lote cambia (p.ej., de soja a trigo en la nueva temporada), la edición lo permite. Esto no afecta históricamente a fumigaciones pasadas (que tal vez asumían cultivo anterior). Es algo a tener en cuenta: un lote podría haber tenido múltiples cultivos a lo largo del uso. La app en MVP no lleva historial de cultivo por temporada; simplemente sobrescribe. Esto está bien, pero el piloto debe saber que la etiqueta cultivo del lote es la actual. Si quisieran registros históricos, sería más complejo (fuera de alcance MVP). - **Edición de geometría:** hay que recalcular hectáreas al finalizar la edición. Usar los mismos métodos que en creación (Turf). Comprobar que la UI actualice el valor mostrado de área en tiempo real si se puede mientras mueve vértices (o al menos al terminar). - **Validación especial:** Si un lote cambia de nombre, verificar (opcional) que no choque con otro ya existente. Podemos decidir si permitimos duplicados; no es crítico técnicamente, pero para piloto podría ser confuso. Podemos dejarlo y quizás en notificación de implementación decir "se recomienda nombres únicos". - **Cancelar edición:** Al cancelar, si se habían hecho cambios parciales (mover un vértice, etc.), idealmente revertir al estado original. Si usamos un formulario controlado con estado inicial, al volver a detalle simplemente recargamos el original. Para geometría, si hizo cambios en el mapa y cancela, habría que descartar. Podría ser tan simple como recargar la página de detalle (descartando ediciones no guardadas). - Probar editando tanto campos textuales solos, polígono solo, y ambos a la vez, para asegurar todo se guarda. - **UX:** Indicar de alguna forma cuando un lote fue editado exitosamente (un toast o simple mensaje al regresar a detalle, aunque no es estrictamente necesario si ve sus cambios reflejados).

Historia de Usuario: Eliminar Lote

Descripción: Como piloto, quiero poder **eliminar un lote** que ya no uso o fue ingresado por error, para mantener mi lista de campos organizada y sin información obsoleta.

Pasos (Actual vs Esperado): 1. **Actual:** Si un campo ya no es relevante (vendido, unificado con otro, etc.) o se ingresó duplicado por error, actualmente permanecería en la lista para siempre, lo que puede causar confusión al registrar aplicaciones.

→ **Esperado:** La app permite al piloto eliminar un lote, quitándolo de la lista y ya no considerándolo en futuras operaciones, manteniendo la información pertinente al día. 2. **Actual:** Un lote no válido en la lista puede llevar a seleccionar el campo equivocado al cargar una aplicación.

→ **Esperado:** Eliminando el lote, se evita esa posibilidad, ya que solo quedan lotes válidos en el sistema.

Criterios de Aceptación: 1. En la vista de detalle de un lote, al pulsar el botón **Eliminar** (usualmente representado con un ícono de papelera), la app muestra una **confirmación** al usuario: "¿Confirma que desea eliminar el lote X? Esta acción no se puede deshacer.". 2. Si el piloto confirma la eliminación: - El lote es removido del almacenamiento local: se elimina su entrada del array de `lotes` en `localStorage`/estado. - En consecuencia, ya no aparece en la **lista de lotes**. - Cualquier referencia a este lote en otras partes de la app debe manejarse: * Por ejemplo, si hay fumigaciones registradas que

apuntaban a este lote, ¿qué sucede? (Ver nota abajo sobre integridad referencial). - La app navega fuera de la vista de detalle (puesto que ese lote ya no existe). Podría volver a la lista de lotes automáticamente tras la eliminación, actualizada sin el elemento. 3. Si el piloto cancela en el diálogo de confirmación, no se elimina nada y permanece en la vista de detalle sin cambios. 4. **Restricción de eliminación:** Si el lote tiene aplicaciones de fumigación asociadas registradas, la app debe evitar su eliminación o al menos advertir de ello. Posible criterio: - *Opción 1:* No permitir eliminar un lote que tenga fumigaciones registradas (para no dejar huérfanos esos registros). Mostrar un error: "No se puede eliminar este lote porque tiene aplicaciones registradas. Elimine o edite esas aplicaciones primero.". - *Opción 2:* Permitir eliminarlo pero conservar las fumigaciones históricas con referencia al lote eliminado. En tal caso, habría que decidir cómo muestran esas fumigaciones su lote (quizá solo el nombre textual guardado en el registro, ya que el id ya no estaría en la lista). Esta opción es más compleja de manejar y puede causar pérdida de integridad. - Para MVP, es **más seguro impedir la eliminación** de lotes que hayan sido usados en al menos una fumigación. Así el historial no se rompe. 5. Entonces, antes de eliminar, la app debe hacer una verificación sencilla: buscar en la lista de fumigaciones si alguna tiene `loteId == id` del lote a borrar. - Si encuentra alguna, cancelar la eliminación y avisar: "No se puede eliminar el lote porque existe al menos una aplicación registrada en él.". - Si no encuentra ninguna, procede con eliminación normalmente. 6. La eliminación ocurre offline en localStorage. En caso de sincronizar posteriormente, se debe marcar para que el servidor también elimine ese lote (p.ej., acumulando la acción de delete).

Tareas Técnicas: - Frontend:

- En **LoteView** (detalle) implementar el botón Eliminar con el diálogo de confirmación. Puede usar un componente modal genérico. - En la confirmación positiva, ejecutar la lógica: * Comprobar antes la condición de referencias: acceder a la lista de fumigaciones local (desde context o localStorage) y filtrar las que tengan `loteId` igual al que se quiere borrar. * Si la lista filtrada no está vacía, abortar: en lugar de borrar, mostrar otro alerta diciendo que no se puede borrar por tener aplicaciones vinculadas (o se podría listar cuántas tiene). * Si está vacío (no hay referencias), proceder a borrar: - Remove el lote del estado global de lotes (ej., `removeLote(id)` en un hook/context). - Actualizar `localStorage['lotes']` con el lote removido. - Navegar de regreso a la pantalla de lista de lotes (posiblemente usando `navigate('/lotes')` u otra lógica). - Opcional: mostrar un toast "Lote eliminado". - Actualizar la UI de lista automáticamente al eliminar (si se usa estado global, sucederá solo; sino, re-leer localStorage al volver a lista). - Si la vista de detalle se accedía a través de router, hay que manejar que tras eliminación ya no hay detalle. Puede hacer `navigate(-1)` para volver atrás (pero si venía del dashboard podría ir a otro lado). Mejor llevarlo explícitamente a `/lotes`.

- **Backend:** - Definir endpoint **DELETE** `/api/lotes/{id}` para eliminar en servidor. En multiusuario se controlaría permisos. En offline, no se ejecuta hasta sync. - Marcar el lote como pendiente de eliminación en local (p.ej., agregar a una cola de deletes a sincronizar). Alternativamente, borrarlo local y confiar en sync que detecte la ausencia y envíe delete, pero implementar eso es complejo. Más fácil: mantener un array `pendingDeletions` con IDs para enviar a server luego. - **Notas de Implementación:**

- **Integridad de datos:** Decidimos no eliminar si hay fumigaciones referenciando. Esto mantiene consistencia. Documentar al usuario (piloto) que primero tendría que borrar los registros de aplicaciones del lote (o editarlos para apuntar a otro) antes de eliminar el lote, si realmente quiere eliminar todo rastro. - Si en el futuro se desea borrar en cascada (borrar lote y sus fumigaciones), habría que confirmar eso con más seriedad ("Esto eliminará también 3 aplicaciones registradas en este lote. ¿Confirma?"). En un contexto profesional, suele preferirse no borrar históricos de fumigación, por normativa, así que mejor no permitir eliminar lotes usados. - **Recuperación:** No habrá papelera ni undo. Una vez eliminado, se pierde. Asegurar esa irreversibilidad en la confirmación para que el usuario esté consciente. - Verificar tras eliminar: la cantidad de lotes en dashboard (si se muestra total de campos) debe decrementar. Igualmente, eliminar un lote no debería afectar conteo de aplicaciones o hectáreas en dashboard, salvo que contaban ese lote en "total fields" y ahora se resta uno (eso sí debería ajustar). - Manejar eliminación de un lote que fue sincronizado con servidor: en offline-first,

podríamos marcarlo y quitarlo local. Si luego el piloto crea otro lote con mismo id (improbable, id suelen ser UUID), nada grave. Al sync, se deberá enviar la orden de borrado. Este caso es más de sync, pero mencionar: conservar quizás el ID en la cola de deletes. - **UX:** Si la app tiene pocos lotes, eliminar puede hacer que la lista quede vacía; tener un mensaje adecuado para lista vacía (como ya mencionado en listar). - Probar eliminar tanto con casos permitidos (sin fumigaciones) como no permitidos (con fumigaciones, donde debe bloquear).

Épica: Registro de Aplicaciones de Fumigación

Historia de Usuario: Listar Aplicaciones de Fumigación

Descripción: Como piloto, quiero ver el **listado de todas mis aplicaciones de fumigación** registradas en la app, para revisar el historial de trabajos realizados y los que tengo planificados, incluso sin conexión.

Pasos (Actual vs Esperado): 1. **Actual:** El piloto llevaría un registro de fumigaciones en papel o en diferentes archivos, dificultando ver rápidamente el historial completo o filtrar por estado.

→ **Esperado:** La aplicación muestra una **lista consolidada de aplicaciones** de fumigación, con posibilidad de filtrar por estado (Planificadas, Completadas, etc.) o buscar, permitiendo al piloto encontrar información de una aplicación pasada o ver qué está pendiente de un vistazo. 2. **Actual:** Ordenar o buscar aplicaciones (por fecha o campo) manualmente es propenso a errores.

→ **Esperado:** La lista en la app está automáticamente ordenada (por ejemplo, por fecha) y ofrece **búsqueda** para localizar una aplicación por nombre de lote, fecha o producto rápidamente.

Criterios de Aceptación: 1. Existe una sección "**Aplicaciones**" (o "Fumigaciones") en la app donde se listan todas las aplicaciones de fumigación registradas. Cada elemento de la lista muestra un resumen: - Fecha de la aplicación (o fecha planificada si futura, o fecha real si ya realizada), - Nombre del lote donde se realizó, - Estado actual (Planificada, En Proceso, Completada, Cancelada), - Opcionalmente, el **nombre/título** de la aplicación si el piloto le asignó uno (por ejemplo "Fumigación Otoño 2025"), - Quizá el tipo de tratamiento o producto principal usado, para reconocimiento rápido. 2. Las aplicaciones en la lista están ordenadas por **fecha**, por defecto de más reciente a más antigua (aplicaciones futuras planificadas podrían mostrarse primero también si se prefiere). Se debe definir claramente: posiblemente mejor ordenar por fecha descendente siempre ³³ ³⁴. 3. El piloto puede **filtrar por estado**: habrá pestañas o un filtro (dropdown) para ver solo "Pendientes" (Planificadas/En Proceso no completadas) o solo "Completadas" o "Canceladas". Esto facilita centrarse en tareas pendientes vs historial. Por ejemplo, botones en la UI: *Todos, Planificados, Completados, Cancelados*. Al tocar uno, la lista se actualiza mostrando solo los registros con ese estado ³⁵ ³⁴. 4. También se implementa una **búsqueda textual** en esta lista. Al ingresar texto en el buscador, se filtran las aplicaciones cuyo lote o producto o título coincida con el texto. Por ejemplo, buscar "Soja" podría encontrar aplicaciones en lotes de soja (si se incluye cultivo en búsqueda) o con producto con soja en nombre (si existiera), pero mínimo debe buscar por nombre de lote y quizás tipo de tratamiento/producto. 5. Cada elemento de la lista es seleccionable; al tocar una aplicación, navega a la **vista de detalle** de esa aplicación (ver historia siguiente). 6. En cada ítem, un **indicador de color** o icono según el estado mejora la visual: por ejemplo, usar un punto o etiqueta de color azul para Planificada, naranja para En Proceso, verde para Completada, rojo para Cancelada ¹². Esto permite con solo mirar la lista identificar qué está hecho y qué no. 7. Función de **eliminar**: se puede permitir eliminar una aplicación directamente desde la lista (como un swipe o botón en el ítem). Sin embargo, puede posponerse a la pantalla de detalle. Si se implementa en lista, debería pedir confirmación. (Esto lo cubriremos en historia "Eliminar Aplicación"). 8. **Offline:** La lista carga de los datos locales (claves `fumigaciones` en localStorage). Si no hay ninguna, mostrar "No hay aplicaciones registradas. Use 'Nueva Aplicación' para agregar.".

Tareas Técnicas: - Frontend:

- Desarrollar componente **FumigacionesList** para renderizar la lista de aplicaciones ³⁶ ²². Puede ser similar a LotesList en estructura: quizás usar un table o list group. Columnas: Fecha, Lote, Estado, (opcional Producto/Tipo). - Formatear la **fecha** en un formato legible (ej. "10/09/2025" en lugar de ISO). Si se incluye hora, quizás no necesario en lista, solo fecha. - Implementar las **pestañas de filtro** de estado: puede ser un conjunto de botones toggle. Por ejemplo, un estado de filtro `filterStatus` que puede ser "ALL", "PLANIFICADA", "EN_PROCESO", "COMPLETADA", "CANCELADA". En el render, filtrar las fumigaciones a mostrar por ese estado (o todos si ALL). - Implementar el **buscador** textual similar a lotes: un input que filtre por ciertos campos. Determinar campos: seguramente por nombre del lote asociado y por producto o título de aplicación. Para facilitar, podría convertir todo a strings (lote nombre + tipo + producto) y buscar substring. - Cada item de la lista debe tener un onClick que `navigate` a `/aplicaciones/{id}` o similar para detalle. - Mostrar los **estados con color**: se puede utilizar un pequeño `<span>` con clase según estado, o una badge textual ("Completada") con background color. Los colores sugeridos en análisis: Planificada=azul, En Proceso=naranja, Completada=verde, Cancelada=rojo ¹². Aplicar esos estilos. - Integrar la lista con la fuente de datos local: usar un hook `useFumigaciones()` análogo a `useLotes`. Este hook podría proporcionar la lista y métodos para filtrar, pero filtrar quizás se hace en el componente mismo con state local. - Asegurar que cuando se añade/edita/elimina una aplicación, la lista refleje los cambios (similar estrategia a lotes: actualizando estado global o refetch localStorage). - (Opcional) Implementar funcionalidad de eliminar en la lista: quizás un icono de papelera en cada fila. onClick, confirmar y borrar (ver historia eliminar aplicación). - Añadir la capacidad de **ordenar** si se quisiera: podría ser predeterminada (descendente por fecha). Si se quisiese permitir orden manual (por nombre, por fecha asc/desc), se podría hacer clic en encabezados. No es prioridad en MVP, pero la estructura de datos (array local) lo permite con sort. - **Backend:**

- Endpoint **GET** `/api/fumigaciones` para obtener todas las fumigaciones. En offline, no se usa directamente. Pero para sync, esto se considerará. Podría soportar query params como `?status=PLANIFICADA` para filtrar por estado en servidor, aunque en local no es necesario porque todo está ya. - No other backend action for just listing, aside from what is needed for sync later. - **Notas de Implementación:**

- **Identificación de aplicación:** En la lista, probablemente la clave principal para el piloto sea la combinación de fecha + lote. Si el piloto distingue aplicaciones por algún nombre, podríamos usar un campo "Nombre de Aplicación" (title) que mencionamos en la historia de registro. Si se implementa, mostrarlo en la lista como título. Si no se implementó título, podríamos mostrar tipo de tratamiento como subtítulo. Ejemplo de item: "10/09/2025 - Campo Los Álamos (Herbicida) - Completada". - **Volumen de datos:** Es probable que haya muchas aplicaciones a lo largo del tiempo. La lista debe manejar potencialmente decenas o cientos de entradas. En MVP, la cantidad es manejable (se pueden paginar en un futuro si crece mucho). Por ahora, cargar todo está bien. Pero considerar performance si se filtra mucho; no es crítico con <500 items. - **Actualización en tiempo real:** Si se marca una aplicación como completada (ver historia cambiar estado) mientras se está en la lista (por ejemplo con un botón in situ), la UI debe actualizar ese item. Si hacemos ese cambio en detalle y regresamos, la lista se refresca. Podría usar context or eventing. Simpler: refetch data on focus of screen. - **Datos mostrados:** Decidir si mostrar fecha planificada vs fecha real: - Para completadas, sería bueno mostrar la fecha en que se completó (si se registró). - Para planificadas, la fecha planificada (futura). - Podríamos usar un campo "fecha" que signifique "fecha de la aplicación (real o prevista)". En el modelo visto, hay `fechaPlanificada` y `fechaRealizada`. Quizá en la lista, si estado < Completada, usar `fechaPlanificada`, si Completada usar `fechaRealizada` (o planificada si no se registró real). - Para no complicar: usar siempre `fechaPlanificada` como referencia (asumiendo en completadas es igual a la realizada o cercana). O almacenar un solo fecha principal en registro (no, mejor mantener distinción internamente). - **Estado "En Proceso":** Depende si lo usamos. Quizás en MVP el piloto no va a marcar "En Proceso" mucho, pero lo incluimos para completitud. Aparece en lista con su color. - **Responsive design:** Asegurar que en pantallas pequeñas se vea bien. Quizá en móvil (teléfono) habría que apilar info en dos líneas por item (fecha + lote en primera línea, estado + producto en segunda, por ej.). En

tablet puede ser en una línea. - **Iconos:** Se pueden usar íconos representativos: un check para completadas, un clock para planificadas, etc., junto al texto de estado o en lugar de color label. Solo si hay tiempo, los colores directos ya comunican. - Probar la búsqueda y filtros combinados: por ejemplo, filtro "Completadas" y buscar "Trigo" debería solo buscar entre completadas. Implementar para que ambos filtros apliquen simultáneamente (filtrar la lista original por estado, luego por texto). - **Default view:** Quizás mostrar por defecto solo pendientes (planificadas/en proceso) para enfocarse en lo próximo, con opción de ver todas. O mostrar todas y que el piloto use filtro. - Podría ser útil arrancar en "Pendientes" tab para que no tenga que filtrar manual cada vez. Pero no especificado; se puede preguntar preferencia, en ausencia, mostrar *Todos* y confiar en visual de colores.

Historia de Usuario: Ver Detalle de una Aplicación

Descripción: Como piloto, quiero ver todos los **detalles de una aplicación de fumigación** específica que registré, para consultar qué trabajo se hizo o está planificado (incluyendo campo, productos, dosis, condiciones) sin tener que recurrir a mis apuntes en papel.

Pasos (Actual vs Esperado): 1. **Actual:** Para recordar o verificar qué se aplicó en cierto campo en una fecha dada (por ejemplo, tipo de agroquímico, dosis, condiciones climáticas), el piloto tendría que buscar en papeles o confiar en la memoria, con riesgo de olvidar algo.

→ **Esperado:** La aplicación proporciona una **vista detallada** por cada registro de fumigación, mostrando toda la información ingresada (campo, fecha, tipo de tratamiento, producto usado, cantidad, piloto responsable, etc.) de forma organizada. 2. **Actual:** En caso de auditorías o simplemente planificación, no hay un lugar centralizado con todos los datos de una fumigación.

→ **Esperado:** Con la vista de detalle, el piloto (o supervisor si hubiera) puede revisar toda la ficha de la fumigación en la app, incluso offline, asegurando transparencia y facilidad de acceso a la información histórica.

Criterios de Aceptación: 1. Al seleccionar una aplicación de la lista, se navega a la **pantalla de Detalle de Aplicación**. En esta pantalla se muestran todos los campos registrados para esa aplicación, incluyendo: - **Lote:** Nombre del lote donde se realizó o se realizará la fumigación. Posiblemente acompañado de algún indicador (ej. un botón "Ver Lote" que permita abrir la vista del lote en mapa). - **Fecha planificada** y, si la aplicación ya se completó y se registró, **Fecha de realización** efectiva. Si la fumigación está completada, ambas podrían mostrarse (p.ej., "Planificada: 10/10/2025 - Realizada: 12/10/2025"). Si no, con solo la planificada es suficiente. - **Estado actual** de la aplicación (Planificada/En Proceso/Completada/Cancelada). - **Tipo de tratamiento:** Preventiva, Correctiva, etc., tal como fue seleccionado. - **Producto aplicado:** el nombre del producto (o categoría, ej. "Herbicida") elegido. - **Dosis:** la cantidad y unidad aplicada (por ejemplo "200 L/ha" o "1.5 kg/ha") si se registró. - **Equipo utilizado:** Avión, Drone, etc., según lo seleccionado. - **Responsable:** nombre del piloto o persona a cargo (puede ser el mismo usuario, pero si se registró campo "responsable" podría usarse). - **Condiciones climáticas:** si se ingresaron, mostrar temperatura, humedad, viento, etc., al momento de la aplicación. - **Coste:** si se registró un costo de la aplicación, mostrarlo (ej. "\$15000"). - **Observaciones:** cualquier nota adicional que el piloto escribió sobre esta aplicación. - Cualquier otro campo que haya sido parte del formulario (por ejemplo, eficacia lograda si lo anotó en observaciones, etc.). 2. La información debe presentarse en secciones claras, por ejemplo: - Sección "Detalles generales": lote, fecha, estado, responsable. - Sección "Tratamiento": tipo, producto, dosis, equipo. - Sección "Condiciones": clima, etc. - Sección "Notas": observaciones. Esto ayuda a la lectura, especialmente en pantalla pequeña. 3. Si la aplicación está **Completada** y tiene una geometría específica distinta al lote (por ejemplo, si en el formulario de fumigación se hubiera dibujado un área de tratamiento específica, aunque en MVP asumimos que usa el área del lote), se podría mostrar un mapa con esa área. Sin embargo, en nuestro diseño probablemente la aplicación referencia directamente un lote completo, así que un mapa podría ser redundante. Podríamos en su lugar ofrecer un botón "Ver mapa del lote" que

abre el lote. 4. La vista detalle ofrece acciones: - **Editar Aplicación**: un botón para modificar este registro (ver historia correspondiente). - **Cambiar Estado** rápidamente (si aplica): por ejemplo, si está Planificada, un botón "Marcar como Completada" (ver historia de cambio de estado). - **Eliminar Aplicación**: botón para borrar este registro (ver historia eliminación). Estas acciones deben estar disponibles según contexto (por ejemplo, no mostrar "Completar" si ya está Completada). 5. Todo debe funcionar offline, ya que los datos están almacenados localmente. Si falta algún detalle que dependiera de servidor (no es el caso, todo fue ingresado local), se indica que no hay datos. 6. Formato de presentación: usar unidades adecuadas (por ejemplo, si dosis se guarda con unidad, mostrar junto; si no, concatenar valor + unidad). 7. (Opcional) Si se calculan métricas como efectividad a posteriori, podría mostrarse aquí, pero eso no se implementó en MVP.

Tareas Técnicas: - Frontend:

- Crear componente **FumigacionView** para mostrar la información detallada ³⁷ ²² . - Obtener el registro de fumigación actual vía ID (similar a lote: buscar en lista global de fumigaciones por id param de URL). - Formatear todos los campos para mostrarlos user-friendly. Por ejemplo, mapear códigos a textos: si `tipoTratamiento` = "PREVENTIVO", mostrar "Preventiva". Si `equipo` = "AVION", mostrar "Avión". - Disponer la info en un layout de secciones: se pueden usar headings pequeños para cada grupo de datos. - Incluir los botones de **Editar**, **Eliminar**, y un control para **Cambio de estado**: * Si estado = Planificada o En Proceso, mostrar botón "Marcar como Completada" y otro "Cancelar Aplicación" (o un menú de acciones de estado). Si estado = Completada o Cancelada, esos botones pueden ocultarse o deshabilitarse (ya no tiene sentido cambiar a otro). * Estos botones invocarán las funciones correspondientes (ver historias). - Si se considera mostrar un mapa pequeño: * Podría ser útil visualizar el lote allí mismo. Se puede embeber un mini-mapa centrado en el lote con un marcador. Sin embargo, dado que en LoteView ya hay el mapa, quizá no duplicar. Mejor un botón "Ver Lote", que al pulsar navegue a LoteView de ese lote (así reutiliza). * Implementar ese botón/link en, por ejemplo, el nombre del lote (hacerlo clickeable). - Agregar también un indicador visual del **estado** (como badge de color) junto al campo estado para consistencia. - **Backend**:

- Endpoint **GET** `/api/fumigaciones/{id}` para obtener detalle de una fumigación. En offline, no hace falta; los datos ya están en el dispositivo. - Asegurar que la estructura de datos de fumigación tiene todo lo necesario. (Según el análisis, el esquema fumigación es muy completo ³⁸ ³⁹ , tenemos más campos de los que quizá usamos en MVP, pero es bueno tenerlos). - No hay más acciones de backend excepto considerar sync de ediciones. - **Notas de Implementación**:

- **Mostrar fechas**: Si hay dos fechas (planificada vs realizada), se pueden mostrar ambas si difieren. Por ejemplo, si planificada != realizada (caso de ejecución tardía), conviene mostrar ambas. Sino, con una es suficiente. Para MVP, podríamos no complicar: mostrar solo una "Fecha" que sea la planificada o la realizada dependiendo del estado (si completada, quizás usar la fechaRealizada; si no completada, usar planificada). Sin embargo, el usuario podría querer saber cuándo estaba programado vs cuándo se hizo en realidad. Si se ingresó la fecha de realización al marcar completada, guardarla, entonces sí mostrar las dos con sus etiquetas. - **Nombres y referencias**: Como ya mencionado, si guardamos solo id de lote en el registro, al mostrar detalle necesitamos buscar el nombre del lote para mostrarlo. Hacer esa referencia (usando lista de lotes global). Alternativamente, podríamos haber guardado redundante el nombre en el registro para snapshot histórico (pero eso complica mantener actual si lote renombrado; sin embargo, es histórico, no necesariamente actualiza). Preferible siempre mostrar nombre actual vía id lookup. - **Dosis y unidades**: En el formulario de fumigación se debería haber capturado dosis y la unidad por separado (ej 200 y l/ha). En la vista, concatenarlos: "200 l/ha". Si no hay dosis registrada, ocultar ese campo. - **Clima**: Similar, si se guardó un objeto condicionesClimaticas, formatear: e.g., "Clima: 25°C, 60% hum, Viento 5 km/h (Moderado)" si esos datos existen. Si no se capturó nada (campos vacíos), se puede omitir la sección o poner "No registrado". - **Notas**: Mostrar texto libre, asegurarse de manejar saltos de línea si existen (e.g., pre-wrap CSS). - **Cambiar estado in situ**: Podría ser útil permitirlo sin ir a editar toda la aplicación. Por eso se plantea un botón para completarla o cancelarla. Implementar esa lógica en la historia siguiente (Cambiar Estado), pero la presencia de los botones se

define aquí. E.g., si se presiona "Marcar Completada", se setea estado a Completada, fechaRealizada = hoy (si se decide auto-fijar), se guarda y se refresca esta misma pantalla. Similar para Cancelar. - **Navegación:** Después de editar o cambiar estado, volver a esta vista actualizada hace sentido. Asegurar refresco. - Si implementamos el "Marcar completada" aquí, hay que decidir qué fecha de realización usar: podríamos abrir un mini-diálogo para que confirme la fecha real (por defecto hoy) o asumir actual. Simplicidad: al marcar completada se pone fechaRealizada = ahora. - **Eliminar:** si se borra desde detalle, después de confirmación debería navegar fuera (quizá a lista). Eso en historia Eliminar Aplicación. - Probar la vista con distintos estados y con campos faltantes (no todas las fumigaciones tendrán todos los campos, e.g. clima puede no haberse llenado). Asegurar que se maneja ausencia (no mostrar "undefined", sino omitir o mostrar "N/A").

Historia de Usuario: Registrar Nueva Aplicación de Fumigación

Descripción: Como piloto, quiero **registrar una nueva aplicación de fumigación** (ya sea planificada o ejecutada) ingresando todos sus detalles (campo, fecha, producto, etc.), de modo que quede documentada digitalmente y disponible offline.

Pasos (Actual vs Esperado): 1. **Actual:** El piloto planifica fumigaciones en un cuaderno o solo de palabra, y luego anota resultados post-vuelo manualmente. Puede olvidar datos o perder registros.

→ **Esperado:** Con la app, el piloto crea un registro para cada fumigación: especifica qué va a hacer o hizo, en qué lote, cuándo, con qué producto, etc., asegurando que la información quede guardada en la app y pueda sincronizarse posteriormente. 2. **Actual:** Sin un sistema central, otros (ej. agrónomo, dueño) no ven en tiempo real la planificación ni historial.

→ **Esperado:** Una vez registrado en la app (y sincronizado, eventualmente), la información podría ser compartida. Pero de momento, al menos el piloto la tiene consolidada offline para consultas.

Criterios de Aceptación: 1. Desde la sección Aplicaciones (o el Dashboard), el piloto puede iniciar la creación de una **Nueva Aplicación** mediante un botón "Nueva Aplicación". 2. Se muestra un formulario para capturar los detalles de la fumigación. Los campos incluyen: - **Lote objetivo:** selección del lote donde se realizará la aplicación. Debe presentarse una lista desplegable con los nombres de los lotes registrados (datos offline) ⁴⁰, o un componente especializado (como un `LoteSelector` con vista previa) ⁴¹. Este campo es obligatorio. Si el lote deseado no está en la lista, quizá ofrece opción "Agregar nuevo lote" como atajo (no obligatorio, pero útil). - **Fecha planificada:** un selector de fecha (y hora, opcional). Si es una aplicación futura, aquí se pone la fecha prevista. Si es un registro retrospectivo (ya hecha), podría poner la fecha en que se realizó. Este campo es obligatorio. (Hora podría ser útil pero no indispensable; MVP puede solo pedir fecha). - **Tipo de tratamiento:** desplegable con opciones *Preventiva, Correctiva, Masiva, Selectiva*, etc. ⁴². Obligatorio. - **Producto a aplicar:** desplegable con catálogo de productos/químicos principales (ej. Insecticida, Fungicida, Herbicida, Acaricida, etc., o nombres comerciales) ⁴³. Podría ser una lista fija de 8 tipos genéricos como sugiere análisis, o permitir texto libre. Al menos obligatorio seleccionar un tipo genérico. - **Equipo a utilizar:** desplegable con opciones *Avión, Drone, Tractor, Manual*, etc. ⁴⁴. Obligatorio. - **Dosis y unidad:** campo numérico para cantidad y un dropdown para unidad (por ejemplo "L/ha" o "Kg/ha"). Opcional pero recomendado si el piloto maneja dosis. Se puede validar que sea número positivo. - **Responsable:** texto (por defecto podría venir el nombre del piloto usuario). Opcional si siempre es él mismo, pero dejémoslo editable por si registra fumigaciones hechas por otro piloto o asistente. - **Condiciones climáticas:** podría ser un sub-sección con campos: * Temperatura (°C, número), * Humedad (%), * Viento (km/h) y quizás dirección o descripción breve del viento ("Moderado", "Fuerte"). Todos opcionales; el piloto los llena si los midió en el momento. - **Costo:** número (en moneda local) si quiere registrar cuánto costó la aplicación (combustible, insumo). Opcional. - **Observaciones:** textarea para notas libres (ej. "Se observó deriva hacia el norte", "Aplicación realizada en 2 pasadas", etc.). Opcional. - **Estado inicial:** este podría determinarse automáticamente: * Si la fecha planificada es futura (posterior a hoy) => Estado por

defecto "Planificada". * Si la fecha es hoy o pasada => quizás el piloto la está registrando tras realizarla, entonces Estado por defecto "Completada". Pero cuidado: puede que esté registrando algo que hizo hoy más temprano, mejor permitir elegir. Para simplicidad, podríamos poner Estado como un dropdown en el formulario con opciones Planificada o Completada (y quizá En Proceso, pero no suele crearse directamente en proceso). O decidir lógica: suponer que todo lo creado se inicia Planificada y el piloto luego la marcará completada. Pero si está ingresando histórico, sería bueno poder indicar completada ya. Tal vez incluimos un campo **Estado** manual en el formulario, con default "Planificada". El piloto podría cambiarlo a "Completada" si ya la hizo, o "Cancelada" si quiere registrar que no se hará. (En MVP, Cancelada quizás no se crea de entrada, sino se cambia después si no se pudo hacer). 3. Validaciones: - Todos los campos obligatorios (Lote, Fecha, Tipo, Producto, Equipo) deben estar llenos; si falta alguno, no se permite guardar y se resaltan los faltantes. - Fecha debe tener formato válido; si incluye hora, validar hora. - Campos numéricos (dosis, clima, costo) si se llenan deben ser numéricos válidos. 4. Al confirmar **Guardar**, se crea un nuevo registro de aplicación: - Se le asigna un `id` único (igual que lotes, preferible UUID). - Campo estado: si incluimos lógica, asignarlo según selección o default (Planificada). - Si estado = Completada al guardar, podríamos setear `fechaRealizada = fecha planificada` o pedirla aparte. Quizá mejor: si elige Completada, asumir la planificada es la realizada (y se puede editar más tarde si difiere). - Armar un objeto con todos los campos y guardarlo en la lista de fumigaciones en `localStorage`. - Actualizar el estado global (context) de fumigaciones para incluirlo. - La app luego navega a algún lado: podría ser directamente a la **vista de detalle de la aplicación** recién creada para revisión, o de vuelta a la lista de aplicaciones. Depende del flujo deseado; mostrar detalle inmediatamente puede ser bueno para confirmar. 5. Si el formulario se cancela, nada se guarda. 6. **Offline**: Todo el proceso es local. Si más tarde hay sincronización, este registro se marcará para enviar al servidor. 7. Idealmente, antes de guardar, se podría ofrecer un **modo de previsualización** (como mencionó el análisis: preview mode de verificación ⁴⁵). Sin embargo, para MVP, con rellenar y revisar en la misma pantalla es suficiente. Podríamos implementar un diálogo de confirmación "¿Confirma guardar esta aplicación?" mostrando un resumen, pero no es estrictamente necesario.

Tareas Técnicas: - Frontend:

- Crear componente **FumigacionForm** para la creación/edición de aplicaciones ³⁶ ²². Debe manejar un estado interno para todos los campos mencionados. - Traer la lista de lotes para llenar el selector de Lote (useLotes context). - Poner valores por defecto razonables: Fecha por defecto = hoy (piloto puede cambiar si planifica futuro). Estado por defecto = Planificada. - Si implementamos estado como campo seleccionable, cargarlo con Planificada default. - Implementar listas fijas: * Tipo tratamiento: [Preventiva, Correctiva, ...] – usar EXACTOS términos que usamos en modelo (e.g., mayúsculas internamente, pero mostrar capitalizado). * Producto: [Insecticida, Fungicida, Herbicida, Acaricida, etc.] (o podría ser un combo autocompletable, pero MVP lista fija es ok). * Equipo: [Avión, Drone, Tractor, Manual, etc.]. * Unidad de dosis: [L/ha, Kg/ha]. - Validar en onSubmit: campos obligatorios presentes. Para visual, se puede usar HTML required o manualmente manejar y mostrar error hints debajo de campos. - Al guardar, construir objeto:

```
{
  id, loteId, fechaPlanificada, fechaRealizada (si estado Completada, podría
= fechaPlanificada o pedida), estado,
  tipoTratamiento, producto, equipo,
  dosis, unidadDosis, responsable, costo,
  condicionesClimaticas: {temp, hum, viento, condicionViento},
  observaciones,
```

```
    createdAt, updatedAt  
  }
```

(Muchos campos, pero se pueden dejar null los no usados). - Guardar en localStorage: similar a lotes, obtener la lista, push, setItem. O utilizar context with dispatch (like useFumigaciones). - Navegación tras guardar: preferiblemente a la vista de detalle de esta nueva aplicación para confirmación visual.

Implementar navigate(`../aplicaciones/{newId}`) por ejemplo. - **Backend:**

- Endpoint **POST** `/api/fumigaciones` para crear nueva fumigación. Similares consideraciones a POST lotes: no llamado en MVP, pero que la arquitectura permita su futura llamada. En offline, marcar la nueva con `syncPending = true`. - (Si se piensan funciones de server, podría calcular algo o notificar, pero no incumbe ahora). - **Notas de Implementación:**

- **Selector de Lote:** si muchos lotes, un combo simple puede ser poco práctico. El análisis menciona un componente `LoteSelector` con preview ⁴¹, posiblemente mostraba mini mapa del lote seleccionado. En MVP podríamos usar solo un `<select>` con nombres, pero un plus sería al elegir un lote, mostrar su tamaño o ubicación en a hint. No indispensable. - **¿Polígono específico** para la aplicación?: En algunas apps, cada aplicación podría tener un área específica (por ejemplo, si solo fumigó parte del lote, dibuja polígono de esa parte). El prototipo sugiere *Geospatial area selection (polygon drawing or field selection)* como parte del form ⁴⁶. Esto implica que en la form de nueva aplicación podría permitir *en lugar de seleccionar un lote completo, dibujar un polígono de área de tratamiento*. Para MVP, esto complica mucho (sería otro dibujo), y normalmente el lote entero es el área. Quizá omitimos este detalle: asumimos la aplicación cubre todo el lote seleccionado, por tanto no pedimos dibujar nada, solo elegimos lote. - Si quisiéramos replicar el prototipo fielmente: se podría ofrecer opción "Usar lote completo" vs "Dibujar área manual". Pero por simplicidad, no implementaremos dibujo aquí. - **Estado inicial:** Decidir cómo implementarlo. Para MVP, podríamos no pedir el estado en el form y asumir "Planificada" siempre al crear. Luego el piloto la marcará completa cuando la termine. Si el piloto está registrando algo ya hecho, es un poco raro forzarlo a crear como Planificada y luego editar a Completada. - Alternativa: incluir un toggle "Marcar como ya realizada" en el form. Si lo activa, entonces estado = Completada y quizá se podría mostrar campo "Fecha de realización" (por defecto igual a planificada). - Para no liar la UI, quizás dejar estado fijo Planificada, y decirle al piloto que luego entre y la marque Completada. Esto es más trabajo para él pero menos confusión en UI. Dado que es MVP single-user, podríamos suponer que registrarás antes o durante, no después. Si la registra después, todavía puede poner la fecha real en planificada y luego marcar completada. - Conscientes de esto, se puede anotar en notas de usuario doc: "Si la aplicación ya se hizo, regístrala igual y luego cámbiele el estado a completada.". - **Autollenado:** Podríamos autollenar Responsable con el nombre del usuario logueado, si esa info está disponible en `user`. El user model tiene `name` ⁴⁷. Si implementamos perfil, sí. Sino, dejarlo manual. - **Clima data integration:** El análisis planeaba integrar APIs clima en futuro ⁴⁸. En MVP no se hace, pero tener campos para manual input es bien. - **Fecha/hora:** Representar internamente en ISO string para facilidad de ordenado. Por ejemplo, si usa date input (which yields yyyy-mm-dd), guardar como "2025-10-10T00:00:00" etc. Horas si implementadas. - **UID consistencia:** Similar a lotes, usar mismo approach (maybe using same uuid library). - Probar creando fumigación con todos datos, con mínimos datos, ver que lista y detalle la muestran bien. - **UX:** Este formulario es extenso. Podría dividirse en pasos (como planearon multi-step wizard en Phase 2 ⁴⁹). MVP lo puede tener scrollable in one page. En tablet es manejable. Organizar con subtítulos ("Campo y Fecha", "Detalles de Tratamiento", "Condiciones", etc.) para no abrumar. - **Autoguardado (draft):** Phase 2 consideró auto-save drafts ⁴⁹. No implementamos en MVP, pero nota: actualmente si el piloto navega atrás accidental, pierde lo escrito. Sería bueno un alert "Los datos no guardados se perderán". Si tiempo, simple confirm on exit if form dirty. - **Sincronización a futuro:** Asegurar que cada registro tenga algún identificador único permanente. Si offline, se genera local. Al sync, puede usarse tal cual (server can accept client-generated UUID).

Historia de Usuario: Editar una Aplicación de Fumigación

Descripción: Como piloto, quiero poder **editar los datos de una aplicación de fumigación** que registré (por ejemplo si cometí un error o si cambiaron las condiciones planificadas), para mantener el registro exacto.

Pasos (Actual vs Esperado): 1. **Actual:** Si se cargó mal un dato (ej. lote equivocado, o fecha, o dosis) o cambió algo (se pospuso la fumigación a otro día), actualmente no hay forma en la app de corregirlo, quedando información incorrecta.

→ **Esperado:** La app permite entrar al formulario de una aplicación existente, prellenado, para modificar campos como lote, fecha, producto, etc., y guardar los cambios, actualizando el registro. 2.

Actual: Los errores no corregidos podrían llevar a decisiones equivocadas o mala información a superiores.

→ **Esperado:** Con la edición, el piloto asegura que el registro de cada aplicación refleja la realidad (por ejemplo, si aplicó 250 L en vez de 200 L puede corregir ese dato).

Criterios de Aceptación: 1. En la vista de detalle de una aplicación, al pulsar **Editar**, se abre el formulario de aplicación (el mismo que para crear) pero mostrando los datos actuales de esa fumigación ya cargados en los campos correspondientes. 2. El piloto puede modificar cualquier campo excepto quizás el ID (que no se muestra) y posiblemente el estado (el estado quizás es mejor cambiarlo con los botones dedicados, pero podríamos permitirlo aquí también). 3. Validaciones iguales a crear: no dejar obligatorios vacíos, etc. Especialmente si cambia de lote, nuevo lote debe existir; si cambia fecha a vacía no permitir. 4. Al guardar, los cambios se aplican al objeto de la fumigación en local: - Actualizar `localStorage['fumigaciones']` con los campos nuevos para ese registro. - Si cambió el lote, en vistas futuras ese registro mostrará el nuevo lote (id referenciado). - Si cambió estado, actualiza y debería reflejarse en la lista (p.ej., si pasó de Planificada a Completada manualmente aquí). - Actualizar `updatedAt` si se lleva, etc. 5. Regresar a la vista de detalle de la aplicación actualizada (o a la lista, pero preferible a detalle para ver cambios). 6. **Offline:** Cambios se guardan local. Si sync, marcar registro como modificado para update a server. 7. Reglas de negocio: - Si la aplicación estaba Completada, se puede permitir editar datos como producto, dosis (tal vez para corregir lo que efectivamente se hizo). Cambiar estado de Completada a Planificada quizás no tenga mucho sentido pero podríamos permitirlo si fue error (pero más coherente usar Cancelar en su lugar). - Si estaba Cancelada, probablemente no se edita salvo error; se podría incluso bloquear edición en canceladas para evitar confusión (depende). - Para MVP, no complicar: permitir editar cualquier campo de cualquier estado, asumiendo el usuario es responsable de la coherencia (ej., no va a cambiar estado a Planificada después de completada normalmente).

Tareas Técnicas: - Frontend:

- Reutilizar **FumigacionForm** en modo edición. Cargar initial state con los valores de la fumigación actual. - Podría deshabilitar el campo Lote si consideramos que una aplicación no debería moverse a otro lote (porque si se equivocó de lote, quizás mejor borrar esa y crear en el correcto? Sin embargo, para ahorrar, permitamos cambiarlo si fue error de registro). - Permitir edición de todos campos. Si se incluye estado en form, mostrar el actual y permitir cambiar. Sino, dejarlo como estaba y manejar cambio de estado con su acción separada. (Podemos optar por no incluir campo estado en form para evitar duplicidad con botones). - En onSubmit, validar y luego en lugar de crear new object, **mutar** el existente: * Encontrar la fumigación por id en la lista global. * Reemplazar sus propiedades con las del form. * Guardar en localStorage (setItem). * Actualizar context state. - Navegar de vuelta a detalle de esta fumigación. - Notificar de éxito (toast "Aplicación actualizada"). - **Backend:**

- Endpoint **PUT** o **PATCH** `/api/fumigaciones/{id}` para actualizar en server. En offline, coleccionar estos cambios para sync. - Si trackeamos `updatedAt`, setearlo. - **Notas de Implementación:**

- **Recalcular cosas:** Si se modifica la fecha planificada y el estado aún Planificada, en la lista su posición

puede cambiar (por orden de fecha). Asegurar la lista se resorteará la próxima vez se renderice (fácil si siempre sorts by date). - **Cambio de lote:** Si la aplicación ya estaba completada en un lote, cambiar de lote es extraño (sería decir que en realidad se hizo en otro). Podría pasar si se registró mal. Lo permitiremos con advertencia de integridad (no hay referencia contraria, así que ok). - **Cambio de estado aquí vs botón:** Podríamos no permitir cambiar estado en form, para no duplicar. Por ejemplo, si quiere marcar completada, usaría el botón, no editar form. Para mantenerlo sencillo: en form, quizás incluimos estado just in case pero aconsejamos usar los botones de flujo. - Omitir estado del form en MVP no afectaría mucho, porque el piloto puede cambiarlo de otra manera. Sin embargo, si se equivocó y quiere marcar cancelada antes del vuelo, actualmente la única forma sería un botón de cancelar (que tenemos). - Podríamos por simplicidad no tener el campo estado en form de edición. Y dejar que el estado solo se cambie via las acciones directas. Esto evita incoherencias (ej., un piloto podría cambiar estado a completado pero no llenar fechaRealizada en el form). - **Conclusión:** Probablemente **no incluir campo Estado en form**. Mantenerlo separado. - **Clima y otros:** El piloto puede editar/añadir datos de clima o dosis después si inicialmente no los tenía. Soportamos eso. - Probar editar normal (cambiar texto), editar para corregir error grande (lote), etc.

Historia de Usuario: Actualizar Estado de una Aplicación (Completar/Cancelar)

Descripción: Como piloto, quiero **actualizar rápidamente el estado** de una aplicación de fumigación (por ejemplo marcarla como Completada después de realizarla, o Cancelarla si no se llevó a cabo) sin tener que editar todos los detalles, para reflejar su situación real de forma ágil.

Pasos (Actual vs Esperado): 1. **Actual:** Tras realizar una fumigación, el piloto tendría que entrar a editar el registro y cambiar algún campo para indicar que se hizo, o simplemente recordar mentalmente que ya está hecha, lo cual no queda documentado formalmente.

→ **Esperado:** La app proporciona un botón específico para **Marcar como Completada** una fumigación que estaba planificada, que al pulsarlo cambia su estado a Completada y registra la fecha de realización (posiblemente la fecha actual), sin pasos adicionales. 2. **Actual:** Si una fumigación planificada se cancela (por clima u otra razón), el piloto lo anotaría en papel aparte, no quedando claro en la app.

→ **Esperado:** Con un botón **Cancelar Aplicación**, el piloto puede indicar que esa fumigación ya no se realizará, cambiando su estado a Cancelada en la app y excluyéndola de pendientes.

Criterios de Aceptación: 1. En la vista de detalle de una aplicación (o incluso desde la lista, pero enfoque en detalle), existen acciones para **Cambiar el Estado:** - Si la aplicación está en estado **Planificada** o **En Proceso**, se muestra la opción de "**Marcar como Completada**". - También la opción "**Cancelar**" (o "Marcar como Cancelada") por si la tarea no se hará. - Si la aplicación ya está **Completada** o **Cancelada**, estas opciones no se muestran (o están deshabilitadas). 2. Al pulsar "Marcar como Completada": - La app podría pedir confirmación breve ("¿Marcar esta aplicación como Completada?") – opcional. - Luego, actualiza el campo estado del registro a **Completada**. - Setea el campo `fechaRealizada` a la fecha actual (date/hora del dispositivo) si no estaba ya. Si el piloto desea poner otra fecha (porque quizás la completó ayer y está marcando hoy), tendría que editar manualmente; pero en el botón rápido asumimos la actual. - Guarda estos cambios en localStorage. - La interfaz se actualiza: en la vista de detalle ahora se ve estado Completada (verde) y en la lista, esa aplicación movería de pendientes a completadas (lo cual puede significar que si hay filtro, desaparece de pendientes, etc.). - El Dashboard incrementa conteo de completadas y decrementa pendientes en tiempo real. 3. Al pulsar "Cancelar Aplicación": - Similar proceso: confirmación ("¿Cancelar esta aplicación? Esto la marcará como no realizada." – aclaratorio). - Cambiar estado a **Cancelada**. - (No hay fechaRealizada en cancelada, podría setearse `fechaRealizada = null` o dejar planificada, no importa mucho, quizás registrar fecha de cancelación en updatedAt). - Guardar en local. UI actualiza: estado rojo Cancelada. - La aplicación ya no se considera pendiente, así que no debe aparecer en vistas de pendientes. 4. Estas acciones **no requieren abrir el formulario completo**, son directas desde la

vista detalle. Si se quisieran hacer desde la lista, podría haber atajos (ej., un check para completar), pero con cuidado de confirmación; no es requerido en MVP. 5. Después de cambiar estado: - Si se completó, quizá se podría ya no permitir editar ciertos campos (no lo haremos estricto, pero en lógica real, uno podría querer bloquear cambios una vez completada, salvo campos como añadir observaciones). - Si se canceló, similar; probablemente no se edita más. Pero MVP no implementa bloqueo, confía en usuario. 6. **Offline:** Operan local. En sync, estos cambios se enviarán como updates. 7. Un estado "En Proceso" no se está gestionando con un botón específico; teóricamente, se podría tener un botón "Iniciar" para marcar en proceso, pero en la práctica un piloto probablemente no va a entrar a la app a marcar "ahora estoy volando". Ese estado puede ser omitido en MVP o cambiado manualmente en edit si quisieran. Así que nos enfocamos en Planificada -> Completada/Cancelada transitions.

Tareas Técnicas: - Frontend:

- En **FumigacionView**, renderizar botones "Completar" y "Cancelar" según estado actual: * Use perhaps a conditional: `if estado in [PLANIFICADA, EN_PROCESO]: show Complete, Cancel; if EN_PROCESO maybe "Completar" still, or pilot may mark in process manually? We can ignore in process scenario or treat similarly to planificada`. * If estado = COMPLETADA or CANCELADA: hide those buttons. - On "Completar" click: * If confirm dialog returns true, proceed: * Update the state in the JavaScript object: `fumigacion.estado = 'COMPLETADA'`. * If `fumigacion.fechaRealizada` is empty or null, set it = now (format as ISO). * Save changes: find the item in `localStorage['fumigaciones']` (or in context state) by id and update it. Use `setItem` to persist. * Use state management to trigger UI refresh (if using context, context update triggers re-render of detail and list). * Possibly navigate or just update in place (the detail view can be updated just by changing a state variable). * Provide a small success feedback (maybe a toast "Marcado como Completado"). - On "Cancelar" click: * Similar steps: set estado = 'CANCELADA', save, update UI. * Perhaps no date field to set; maybe set fechaRealizada = null (or could set a separate canceledAt, but unnecessary). * Feedback "Aplicación cancelada". - In the **lista de aplicaciones**: * Ensure that next time it renders, it reflects the updated statuses. If we manage via context state, the list should re-filter out the canceled or completed from pending view automatically. Might require calling a re-filter function or using reactive data. * The dashboard stats: if we store those as derived from context, they update automatically. If not, we might recalc manually after state change. Possibly simplest: after marking completed, call a function to recalc dashboard or if using context for fumigaciones list and the dashboard pulls from that, it updates automatically. - One nuance: If "Completar" sets fechaRealizada = now, but if pilot did it at a later time, the date might not match actual event time. We assume that's fine or they can edit the date if they care. - **Backend:**

- Perhaps provide endpoints **PATCH** `/api/fumigaciones/{id}/complete` and **PATCH** `/api/fumigaciones/{id}/cancel` for a future API for convenience. Or simply a general update endpoint used with changed status. - In offline context, mark the record as needing update (with new status). - If multi-user scenario, might notify others or move tasks from pending to done list, but not in MVP. -

Notas de Implementación:

- **En Proceso:** Decidimos no implementar un flujo para En Proceso. Si se quisiera, se podría hacerlo con un botón "Iniciar" que pone estado = EN_PROCESO, y luego el de "Completar" lo ven como siguiente paso. Pero dado un solo piloto y app offline, marcar "en vuelo ahora" no aporta mucho funcionalmente. Lo omitimos o dejemos que si realmente quisieran usarlo, podrían manualmente cambiar en form de editar (no hay campo estado en form, así que realmente no). - **Dashboard pendiente:** Marcar completada reduce pendientes count by one, aumenta completadas. Sin backend, recalcular local. - **Confirmaciones:** Son recomendables para evitar toques accidentales. Implementar un simple `window.confirm` o un modal styled. - **No deshacer fácil:** Una vez marcado completado o cancelado, si se hizo erróneamente, el piloto podría técnicamente editar el estado volviendo a Planificada usando edit (aunque no tenemos campo estado en edit form; alternatively, quizá permitir "revertir" en algún debug way, pero no planificado). Podríamos permitir editar estado en form para manejar revertir. O que cancele y cree nuevo. Documentar: en caso de error, se puede duplicar la aplicación. No ideal. -

Para MVP, asunción: no se revertirá a Planificada, así que confirm necesaria. - **UI feedback:** Al marcar completada, quizás navegar de la vista detalle de esa aplicación de vuelta a la lista de pendientes? Pero mejor quedarse en detalle actualizado, así el piloto puede review y maybe add observation post, etc. - **Auto-add observation?** A veces al completar podrían pedir ingresar alguna nota (ej. rendimiento, problemas). No en scope de quick action, user can edit afterwards if needed. - **Cancelling:** Cancelada presumably means it won't happen. We keep record for history. These might clutter the list unless filtered out by default. If we have filter toggles, we handle it. - **List UI after:** If user had filter = "Pendientes" on list and goes to detail and marks completed and returns (back navigation or after saving?), that item should vanish from pending filter. Possibly need to refresh list view on back navigation. If using React Router, maybe a useEffect to refresh data on focus. - **Atomic update:** The update is simple enough to not worry about conflicts (since single user). If multi, would need check concurrency (not now). - Test transitions: mark completed then try to mark canceled (should not show cancel after completed, so ensure logic hides it once state changed). - If an application was canceled and pilot later wants re-activate it, currently no direct way (other than editing database or re-creating). This likely fine.

Historia de Usuario: Eliminar Aplicación de Fumigación

Descripción: Como piloto, quiero poder **eliminar un registro de una aplicación de fumigación** si fue creado por error o es duplicado, para mantener mi historial limpio y correcto.

Pasos (Actual vs Esperado): 1. **Actual:** Si se ingresó una aplicación por equivocación (ej. lote incorrecto y se volverá a cargar bien, o un registro de prueba), actualmente quedaría en la lista sin poder quitarse, generando ruido en la información.

→ **Esperado:** La app ofrece la opción de **borrar** una aplicación del registro. El piloto puede eliminar definitivamente una entrada de fumigación que no deba estar, reduciendo confusión. 2. **Actual:** Datos incorrectos permanecerían y podrían confundir en estadísticas (ej., contaría en totales).

→ **Esperado:** Al eliminar la aplicación errónea, desaparece de listas y ya no influye en conteos, reflejando datos más precisos.

Criterios de Aceptación: 1. En la vista de detalle de una aplicación (o en la lista, si se implementa ahí), existe un botón **Eliminar**. 2. Al pulsarlo, se pide confirmación: "¿Confirma eliminar este registro de aplicación? Esta acción es irreversible.". 3. Si se confirma, entonces: - El registro de fumigación se elimina del almacenamiento local: se remueve de la lista `fumigaciones`. - Se actualiza la interfaz: * En la lista de aplicaciones, ya no aparece. * Si estaba en estado completada, se decrementa el conteo de completadas en Dashboard (similar para planificada decrementa pendientes). * Si el usuario estaba viendo su detalle, al borrarlo debe navegar fuera (por ejemplo, volver a la lista automáticamente tras confirmar). - (Si se cancela la confirmación, no pasa nada). 4. No se imponen restricciones especiales para borrar aplicaciones, más allá de la confirmación del usuario. Borrar una aplicación no afecta a otros datos (el lote asociado sigue existiendo, etc.), así que es seguro siempre que el usuario entienda que pierde ese registro. 5. **Offline:** Borrado local. En una futura sync, se enviaría un comando de delete de ese registro al server para eliminarlo allá también. 6. Ejemplos de uso: piloto duplicó un registro, o hizo un registro de prueba, o una aplicación planificada fue cancelada y prefiere borrarla en lugar de dejarla marcada cancelada (depende de políticas, pero se lo permitimos).

Tareas Técnicas: - Frontend:

- En **FumigacionView**, implementar el botón Eliminar con su confirm. - Al confirmar: * Identificar el registro actual (tenemos su id). * Removerlo de `localStorage['fumigaciones']`: cargar array, filter out by id, `setItem` con array resultante. * Actualizar estado global (if context used, use something como `removeFumigacion(id)`). * Navegar a la lista de aplicaciones (e.g., `navigate('/aplicaciones')`). * Mostrar un mensaje tipo "Aplicación eliminada" en la lista o toast. - En la **lista de**

aplicaciones, si implementamos botón borrar en cada fila: * Hacer similar confirm y removal logic. (Puede ser extra, pero ya que se hace en detalle, la funcionalidad es la misma). * Si se hace en lista, cuidar que tras borrar se actualicen totales. Podría delegar a la misma remove function. - Asegurar que Dashboard recalcula: la forma más fácil es recal on the fly from data whenever needed. Si context is updated, and if dashboard gets values by computing from context fumigaciones list, it's automatic. -

Backend:

- Endpoint **DELETE** `/api/fumigaciones/{id}` in future for server to remove. Not used now, but mark this deletion for sync. - If the record never existed on server (because sync hadn't occurred), then the sync logic should just avoid sending it. If it existed, send deletion. - **Notas de Implementación:**

- **Irreversibilidad:** Una vez borrado, la única manera de recuperarlo sería reingresándolo manualmente (no hay papelera ni undo en MVP). Asegurar que el usuario es consciente con confirm dialog. - **Efectos en métricas:** El dashboard contaba esta aplicación? * Si estaba completada, se restará del conteo completadas, y hectáreas tal vez (depende si contábamos sum of area por completadas; si sí, recalcular). * Si estaba planificada, restará de pendientes. * Si estaba cancelada, quizás no contaba en ninguno excepto total apps, que también se puede trackear. * En general, recalcular todo es más fiable que decrementar selectivamente, para no cometer error. Por eso conviene calcular metrics from list each time. - **Eliminar vs Cancelar:** Pueden preguntar: "¿Por qué eliminar si puedo cancelar?". Eliminar es para registros incorrectos; Cancelar es para registros válidos que no se ejecutarán pero quieres dejar constancia. Esto se puede documentar en capacitación, pero no es en la app per se. * Podríamos impedir eliminar aplicaciones Completadas para conservar historial (dependerá de políticas, pero en MVP damos libertad). * Quizá un piloto no debería borrar una completada real, pero si fue un duplicado, podría. No ponemos restricciones adicionales en MVP. - **Consistencia con lote:** Borrar una aplicación no tiene el problema de orfandad que tenía borrar un lote, así que no hay checks de ese tipo. - Si la app soportará multi-user en futuro, probablemente borrar fumigaciones sería restringido (pero fuera de alcance). - Probar borrar en distintos estados. Todo similar, no hay dependencia. - UI: tras borrar, lista de aplicaciones debería reflejar de inmediato. Si navigate to list, ensure that list component either reloads from storage or is watching context changes. Possibly useEffect on list mount to refresh from localStorage or rely on context.

Nota final: Todas las funcionalidades descritas arriba son **offline-first**, almacenando los datos en la memoria y en el almacenamiento local del dispositivo (por ejemplo, localStorage del navegador o almacenamiento local del móvil) para disponibilidad sin conexión ⁶. La sincronización con un servidor (cuando se implemente) utilizará esos datos locales como fuente de verdad y enviará cambios acumulados ⁷. En este MVP, se ha priorizado que cada acción del piloto sea inmediata y no dependa de internet. Cada historia incluye los mecanismos para persistir localmente la información y considera cómo se verá afectada cuando posteriormente se conecte y sincronice.

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29
30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49

TECHNICAL_ANALYSIS_REPORT.md

file:///file_00000000553c71f5aae7adfdf038de77